

BackendNotes

Table of Contents

12-factors

auth

buffers

ci-cd

client-server

config-management

console-errors

databases

design-patterns

docker-notas

docker

event-loop

express

file-system

fundamentals

graphql

jenkins

logging-debugging

modules

networking

os-child-process

package-json

path

relational-databases

repl-emitters

rest-api

streams

testing

Production-Ready Node.js Applications

The Twelve Factors

"The Twelve Factors" is a methodology for building modern and maintainable applications. It aims to help developers build applications in the cloud that are portable, scalable, and resilient to changes.

1. **Codebase:** A Node.js app should have a single codebase, tracked in a version control system (VCS) such as Git. This ensures that there is a single source of truth for the application's code. This also gives you an ability to make as many deployments or delivery pipelines as it's needed without worrying about unexpected differences between various sources. Obviously, this is the simplest factor, which does not require any further explanation.
2. **Dependencies:** The dependencies of a Node.js app should be explicitly declared and isolated from the application code. In Node.js it's achieved by a package manager such as npm or yarn, which can manage and install dependencies for the application, as well as perform other handy stuff.
<https://docs.npmjs.com/cli/v9/configuring-npm/package-json>
<https://docs.npmjs.com/cli/v9/configuring-npm/package-lock-json>
<https://docs.npmjs.com/cli/v9/configuring-npm/npm-shrinkwrap-json>
3. **Config:** Configuration settings of your Node.js app (as you can suspect, all those principles apply to any application back-end, even for serverless) should be stored in environment variables, rather than hardcoded in the application code. For example, instead of keeping (hard-coding) a database URL in the application code, it should be stored as an environment variable that can be easily changed without modifying the code. In Node.js we have a lot of tools like dotenv or config to manage configuration through environment variables.
4. **Backing services:** When you design a new Node.js application, it should treat all external services (such as databases, caches, and message brokers) as attached resources that can be accessed through a URL or other connection string. This ensures that the application is decoupled from the specific implementation details of the services it uses. Again, store connection details in the environment and abstract them in your code using libraries like pg for PostgreSQL, redis for Redis, and amqpplib for RabbitMQ. This also relates to external APIs. In this case you need to write your custom code to manage data to and from external APIs pursuing loose coupling between the services. Moreover, in some cases you can extract an independent npm package which will handle all the implementation details and accept a connection string to work with an API.

5. Build, release, run: While building your app you should have a clear separation between the build, release, and run stages of its lifecycle. There are a lot of build tools such as Webpack, Babel, tslib or esbuild to compile the application code, and a deployment tool such as Docker or Kubernetes to package and deploy the application.
6. Processes: You should design a stateless and scalable Node.js application, so that it can be run on multiple instances to handle incoming requests (in other words your app should be horizontally scalable). You can achieve it by using a process manager such as PM2 to manage the application instances or Kubernetes for orchestrating services. In all cases you need some kind of load balancer to distribute traffic across multiple instances within a single compute instance (server or virtual machine) or hosted on different/individual machines. Every instance should independently handle incoming request and do not rely on its internal/cached state.
7. Port binding: A Node.js application should be bind to a port specified by the environment variable, rather than hard-coding a specific port number in your app. This allows the application to be run on any given port number without modifying the code. To some extent this factor seems very similar to the Config factor, but it focuses only on app's port number, as it is important to run your app on a particular or available port to start handling request. If for some reason the port is used, you need to have an ability to change this configuration without doing a code change and additional deployment cycle.
8. Concurrency: You should design your Node.js architecture to handle concurrent requests and avoid blocking operations. Use asynchronous functions and callbacks, and offload blocking operations to worker threads or child processes. In cloud environments there are many ways of achieving concurrency: running multiple app instances and distributing traffic with a load balancer, using function as a service (FaaS) tools, using decoupled or event-driven architectures and related tools, etc.
9. Disposability: Your Node.js apps should be disposable, so that they can be started and stopped quickly and easily. You can use a process manager such as PM2 to start and stop application instances. Or you can configure auto-scaling capabilities in a cloud to automatically run and terminate application instances based on app's load. Your app must be ready to start quickly and terminate gracefully without interrupting existing connections and breaking apps data. Hence, it's important to have a graceful shutdown implemented in your app. In Node.js you can use libraries like graceful-fs and/or similar to handle file system operations gracefully. The implementation of graceful shutdown will also be reviewed in depth soon.

Pets VS Cattle

Pets (Cat): In traditional server management, servers are often treated as pets. This means that each server is unique, receives individual attention, and is manually maintained. Similar to how people care for their pet cats, they are given names, specialized configurations, and are often fixed when they encounter issues. This approach tends to be more time-consuming, less scalable, and prone to human error.

Cattle: The "cattle" approach treats servers as a herd, where they are seen as interchangeable and expendable resources. Instead of giving them unique names and configurations, servers are assigned identifiers or numbers. If a server becomes unhealthy or fails, it is replaced rather than spending time trying to fix it. This approach emphasizes automation, scalability, and resilience.

10. **Dev/prod parity:** You should design a Node.js app to have parity between the development and production environments, so that the same codebase can be run in both environments without modification. This can be achieved by using environment variables to configure the application, and by using a continuous integration and deployment (CI/CD) pipeline to automate the deployment process. This also means that development, staging, and production are similar in terms of hardware configuration. In a real world it can be quite expensive to have several environments (especially development) running on the hardware which is similar to production. Anyway, it is beneficial to have at least one environment similar to production one. You can use various configuration management tools like Ansible, Chef, or Terraform to manage infrastructure in an automated and standardized way.
11. **Logs:** An application should log all its events and errors, so that they can be analyzed and monitored. A logging framework such as Winston or Bunyan can be used for implementing this factor. To aggregate and analyse logs data you should send it to a centralized log management system such as ELK or Splunk. It may be obvious, but it is another important aspect of any modern app.
12. **Admin processes:** In your Node.js apps you should provide administrative interfaces for tasks such as database migrations and system monitoring. You can fulfill this factor by exposing administrative endpoints over HTTP or through a command-line interface (using Node.js REPL or npm scripts to run tasks.).

15-factor cloud-native applications

This new 15-factor app methodology is loosely based around the original Twelve factors (with additional revisions to each of them), but the most significant difference is the inclusion of three new factors:

- API first

- Telemetry
- Authentication and authorization

API-first

When implementing the API-first principle in Node.js, there are several key steps you can follow:

- **Design the API:** Begin by designing the API endpoints, request/response structures, and overall functionality. Determine the resources and operations that the API will expose. Tools like OpenAPI Specification (OAS) or API Blueprint can be helpful for designing and documenting your API.
- **Build the API contract:** Once the API design is ready, create a contract that defines the expected behavior of the API. This contract can be in the form of an OpenAPI document, which specifies the endpoints, input parameters, response formats, and other details.
- **Generate server code:** Use tools like Swagger Codegen or OpenAPI Generator to generate the server-side code scaffolding based on the API contract. These tools can generate Node.js code that sets up the basic structure of the API, including routing, request handling, and error management.
- **Implement API endpoints:** With the generated code as a starting point, implement the API endpoints by adding the necessary business logic and data access layers. This involves writing Node.js code to handle incoming requests, process the data, and generate appropriate responses.
- **Validate and test the API:** Ensure that the API implementation adheres to the defined API contract. Validate the requests and responses against the expected structures and perform thorough testing to ensure the API functions as intended. Tools like Jest, Mocha, or Supertest can assist with testing the API endpoints in Node.js.
- **Document the API:** Documenting the API is crucial for its effective usage. Generate API documentation automatically from the API contract or use tools like Swagger UI (there are a lot of framework-specific packages, e.g., nest-swagger) or Redoc to create interactive API documentation. This documentation helps developers understand how to interact with the API and its available endpoints.
- **Enable client development:** Once the API is implemented and documented, clients can start using it to build applications. Clients can be developed in any programming language or framework that supports making HTTP requests. By focusing on the API-first principle, you provide a clear and consistent interface for client developers to interact with.

Telemetry

Although logging is an important element to building cloud-native applications, it is generally a tool used during development to diagnose errors and code flows. Logging is typically oriented around the internal structure of your app, rather than reflecting real-world customer usage. Telemetry, on the other hand, is focused on data collection once the app is released into the wild. Telemetry and real-time app monitoring enable developers to monitor their application's performance, health, and key metrics in this complicated and highly distributed environment.

Instrumentation. Instrumentation is the process of adding code to your Node.js application to collect relevant data. This can include metrics, logs, and traces. In Node.js, you can use various libraries and tools for instrumentation, such as Prometheus, StatsD, or the built-in console module.

1. **Define metrics:** Identify the metrics that are important for monitoring and understanding your Node.js application. These metrics can include CPU usage, memory consumption, request/response times, error rates, and any other relevant data points. Determine what information will help you assess the performance, stability, and usage of your application.
2. **Integrate monitoring tools:** Choose and integrate monitoring tools that are compatible with Node.js. Popular choices include Prometheus, Grafana, New Relic, Datadog, or the Elastic Stack (Elasticsearch, Logstash, and Kibana). These tools provide features for data collection, visualization, alerting, and analysis.
3. **Collect and store data:** Configure your instrumentation libraries and tools to collect the defined metrics and store the data in a suitable backend, such as a time-series database or log storage. This allows you to retain historical data and perform analysis over time.
4. **Visualize and analyze data:** Utilize the capabilities of your monitoring tools to visualize and analyze the collected data. Create dashboards and visualizations that display the metrics in a meaningful way. This helps you gain insights into the performance of your Node.js application, identify trends, and detect anomalies.
5. **Set up alerts:** Define thresholds and rules to trigger alerts when certain metrics exceed predefined thresholds or when specific events occur. This enables you to proactively respond to critical issues or performance degradation in your Node.js application.
6. **Continuous improvement:** Regularly review and analyze the telemetry data to identify areas for improvement. Use the insights gained from telemetry to optimize performance, identify bottlenecks, fix bugs, and enhance the user experience. Continuously iterate on your Node.js application based on the information gathered through telemetry.

Authentication and Authorization

The addition of the Authentication and authorization factor adds in an important emphasis on security for cloud-native applications. Deploying applications in a cloud environment means that applications can be transported across many data centers worldwide, executed within multiple containers, and accessed by an almost unlimited number of clients. So, it's vital that security is not an afterthought for cloud-native applications and is a very important factor to consider.

Authorization and Authentication

What is authentication

Authentication is the process of verifying that user or device allowing access to a system or resources. In other words, when a user attempts to access information on a network, they must provide secret credentials to prove their identity.

Authentication factors

There are three main categories of credentials used to authenticate or verify identity:

- knowledge factor (the most common factor): verify identity by confirming users through confidential information they have, such as a login and password
- possession factor: verify their identity with a unique object such as secret key
- inherence factor: verify identity through inherent biometric characteristics of the user such a fingerprint, TouchID, FaceID, etc.

Single-Factor Authentication

Single-factor authentication (SFA) or one-factor authentication is the simplest form of authentication method. This method involves matching one credential to verify access to a system (i.e., a username and a password). SFA is considered low-security and recently was added to list of Bad Practices.

Two-Factor Authentication

Two-factor authentication (2FA) provides a second layer of protection resources. 2FA requires something user can prove that he is who he is (e.g., a security token, TouchID). this method is more secure because even if a user's password is stolen, the hacker will have to provide a second form of authentication to gain access.

Three-Factor Authentication

Three-factor authentication (3FA) requires identity-confirming credentials from three separate authentication factors.

Multi-Factor Authentication

Any process that requires two or more factors of authentication known as multi-factor authentication (MFA).

Single Sign-On Authentication

Single sign-on (SSO) authentication allows users to access multiple accounts and applications using just one set of credentials. As an example we can consider Google, which allow users to sign in to other applications using their Google credentials. Basically, applications outsource the authentication process to a trusted third party (such as Google), which has already confirmed the user's identity.

One-Time Password

A one-time password (OTP) is an auto-generated password that is valid for one login session and often used for MFA. For instance, a user will start to log using username and password, which then triggers the application to send an OTP to their registered phone or email. The user can use this code to complete the authentication.

Passwordless Authentication

This method doesn't require any knowledge-based authentication factor. Usually, after entering ID, user will be prompted to authenticate through a registered device or token. SSO and MFA usually use passwordless authentication to improve the user experience and strengthen security.

Certificate-Based Authentication

Certificate-based authentication (CBA) uses a digital certificate to identify a user or device. This certificate, also known as a public-key certificate, is an electronic document that stores the public key data, including information about the key, its owner, and the digital signature verifying the identity. CBA is often used as part of a two-factor or multi-factor authentication process.

What is authorization

Authorization is the process of verification that authenticated user or device has access to the specific resource or functionality.

Authorization strategies

Role-based access control (RBAC)

In this strategy, every user is assigned one or more predetermined roles, and each role comes with a specified set of permissions.

TEXT

```
app.post(`/products`, (req, res) => {
  const user = req.currentUser;
  if (user.role !== Roles.ADMIN || user.role !== Roles.MANAGER) {
    res.status(403).send("User don't have permissions to create products")
  }
  // Logic
  ...
});
```

Attribute-based access control (ABAC)

To cope with unsolvable problems within RBAC, was developed another solution based on attributes. This strategy uses attributes of the resource and user instead of user role to allow or decline access to the resource.

For example, user can create order in the shop, this will add owner attribute to the order entity. In case if someone decided to delete this order by id, we can create function to check that attribute owner of the order is equal to attribute id of the user.

TEXT

```
app.delete(`/order/:orderId`, async (req, res) => {
  const { orderId } = req.params;
  const order = await Order.findOne({ orderId });
  const user = req.currentUser;
  if (order.owner !== user.id) {
    res.status(401).send("User don't have permissions to delete order")
  }
  // Logic
  ...
});
```

Access-control list (ACL)

In this strategy we use list of rules that allow or deny access to the resource.

Using our example with the shop, we can create groups named product_manager and customer that has different scope of permissions to the resources. Now we can assign user to relevant groups and then based on list of permissions allow or deny access to the resource:

TEXT

```
const isAccessAllowed = (resourceName, method, permissions = []) => {
  return permissions.some(perm => {
    return perm.resource = resourceName
      && perm.methods.contains(method)
      && perm.access = 'allow';
  });
}

app.post(`/products`, async (req, res) => {
  const user = req.currentUser;
  const isAllowed = isAccessAllowed('/products/*', 'POST', req.currentUser.permissions);
  if (!isAllowed) {
    res.status(401).send("User don't have permissions to create products");
  }
  // Logic
  ...
});
```

Basic authentication

Basic authentication is an authentication scheme built into the HTTP protocol and usually implemented on the web-server side. When using basic authentication, the client include the authorization header of each request it makes. This header should start with Basic word followed by a space and a base64-encoded string username:password.

As an example, we can consider user with login testuser and password testpassword. To make HTTP request with basic authentication, we need to encode string testuser:testpassword to base64 and add it to authorization header like this:

TEXT

```
Authorization: Basic dGVzdHVzZXI6dGVzdHBhc3N3b3Jk
```

JSON Web Token

JSON Web Token (JWT) is an open standard that provide compact and self-contained way for securely information between components of the system as a JSON object that is digitally signed. JWT can be signed using a secret or a public/private key pair.

Authentication is the most common scenario for using JWT. Once the user is logged in, each subsequent request will include the JWT, allowing the user to access routes, services, and resources that are permitted with that token.

Token structure

- Header
- Payload
- Signature

The header is json object that consists of two parts: the token type (JWT), and the signing algorithm being used:

TEXT

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

The second part of the token is the payload, which contains the claims. Claims are statements about an entity (user, client) and additional data. There are three types of claims:

- registered - set of predefined claims which are not mandatory but recommended, such as iss (issuer), exp (expiration time), sub (subject)
- public - can be defined at will by those using JWTs
- private - custom claims created to share information between components of the system

TEXT

```
{
  "iss": "CloudX Team",
  "sub": "auth",
  "exp": 1505467756869,
  "userId": "b08f86fa-35ba-57f3-8f1b-ads3904660bd",
  "displayName": "John Doe",
  "role": "admin"
}
```

To create the signature part you have to take the encoded header, the encoded payload, a secret, the algorithm specified in the header, and sign that. For NodeJS we can use jsonwebtoken module:

TEXT

```
const jwt = require('jsonwebtoken');
const TOKEN_KEY = process.env.TOKEN_KEY;

const token = jwt.sign(
  {
    userId: "b08f86fa-35ba-57f3-8f1b-ads3904660bd",
    "displayName": "John Doe",
    "role": "admin"
  },
  TOKEN_KEY,
  {
    expiresIn: "2h",
  }
);
```

As you can see, we use `process.env.TOKEN_KEY` to provide secret key that we use to sign the token. This key also will be used to get data from the token and verify if this token valid:

TEXT

```
const jwt = require('jsonwebtoken');
const TOKEN_KEY = process.env.TOKEN_KEY;

try {
  var payload = jwt.verify(token, TOKEN_KEY);
} catch(err) {
  console.error(`Token is invalid`);
}
```

OAuth (Optional)

Open Authorization or OAuth 2.0 is an authorization protocol designed to allow a website or an application to access resources hosted by other web apps on behalf of a user.

This protocol prevents the user from entering his password out of the service provider: the whole process is curtailed to clicking the «I agree to provide access to ...» button. The main approach is to have one secure account, the user can use it for identity verification on other services, without disclosing his password. OAuth also can be used for authorization as well (it allows to provide the rights for the actions that the service client will use for access verification).

There are multiple trusted services such as Google, Amazon, etc. that provide OAuth. From Architecture point of view it allow us to design system using trusted service for

authorization that relieves us of security responsibilities.

Roles

The idea of roles is part of the core specification of the OAuth 2.0. The OAuth 2.0 provide the following:

- resource owner - the user or system that owns the protected resources and can grant access to them
- client - The client is the system that requires access to the protected resources
- authorization server - the server that receives requests from the client for access token
- resource server - the server that protects the user's resources and receives access requests from the client

Identity providers (Optional)

An identity provider (IdP) is a component of the system that provides an end user or device with a single set of login credentials across multiple applications.

As an example, considered log to application using user Google Account. In this example, Google Sign-In is the identity provider.

How do identity providers work?

IdPs communicates using languages like Security Assertion Markup Language (SAML) or data formats like Open Authorization (OAuth). Using an identity provider is convenient for users because it means they no longer must remember multiple logins.

From the service provider's point of view, this approach can be more secure for the following reasons:

- provides a central audit trail of all access events
- saves users the hassle of creating and maintaining multiple usernames and passwords with single sign-on (SSO)
- client application doesn't have to be concerned about protecting personally identifiable information (PII)

Types of identity providers

An enterprise based identity provider used in a corporate enterprise for identity and access management (IAM).

Popular companies that offer IdP services include the following:

- Google
- Facebook

- Apple
- Microsoft
- Amazon Web Services (AWS)

Buffers

Buffer is a special type of object that can store raw binary data. A buffer represents a chunk of memory - typically RAM - allocated in your computer. Once set, the size of a buffer cannot be changed.

- Buffers act like arrays – they are indexable and iterable.
- By default, when you allocate a Buffer, it is filled with zeros (unless you choose otherwise).
- Buffers in Node.js are represented in hexadecimal (hex) by default when you log or inspect them.

Binary data: Data that comes from files, network operations, streams, etc.

Fixed Size: Buffers have a fixed size. If you need a bigger one, you have to create a new buffer and copy data manually.

A Buffer byte holds 8 bits:

TEXT

[1 1 1 1 1 1 1 1] → 8 bits

Each bit is either 0 or 1. So the largest value you can store with 8 bits (when all are 1s) is:

TEXT

Binary: 11111111
Decimal: 255
Hex: 0xFF

UInt8: 0 to 255 (Unsigned byte, default in Buffer) Int8: -128 to 127 (Signed byte, two's complement)

Allocating large buffers

TEXT

```
const buff = Buffer.alloc(0.5e9);
setInterval(() => {
  buff.fill(0x22);
}, 5000);
```

Think of it like this: Imagine you're moving water from a big tank (like a file or internet connection) to small cups (your program). A buffer is like a bucket you use to carry the water in between - it holds the water temporarily while you move it.

A buffer in Node.js looks like this:

TEXT

```
<Buffer 68 65 6C 6C 6F>
```

Working with buffer

Creating a buffer

`Buffer.from()` accepts a string, an array, an `ArrayBuffer`, or another buffer instance. Depending on which params you pass, `Buffer.from()` will create a buffer in a slightly different way.

When passing a string, a new buffer object will be created containing that string. By default, it will parse your input using utf-8 as the encoding

TEXT

```
const buffer = Buffer.from("chihuahua");  
console.log(buffer); // <Buffer 43 68 69 68 75 61 68 75 61>
```

Reading a buffer

TEXT

```
const buffer = Buffer.from("chihuahua");  
console.log(buffer.toString()); // chihuahua
```

When to Use Buffers in Node.js

1. Working with Binary Data

- File I/O operations (reading/writing binary files)
- Network protocols (TCP/UDP raw data)
- Binary data processing (image/video/audio manipulation)

2. Cryptographic Operations

- Storing encryption keys securely
- Hashing algorithms
- Random number generation

3. Network Communications

- Raw socket operations
- Protocol implementations (WebSockets, custom protocols)

Warnings & Notes

1. Memory Usage 500MB isn't small. If you run many large buffers or multiple Node.js processes, you may run into:
 - GC slowdowns
 - Process crashes
 - Out-of-memory (OOM) errors
2. Node.js Heap Size Buffers are allocated outside of the V8 heap (in native memory). So even if your JS heap is small (e.g., 512MB), you can still use large Buffers — but memory pressure can still crash the app.
3. CPU Usage Filling 500MB with a byte every 5s may consume noticeable CPU, especially on lower-end machines.

allocUnsafe

`Buffer.allocUnsafe(size)` creates a new Buffer of the specified size without filling it with zeros or initializing the memory. This makes it faster than `Buffer.alloc()` but potentially unsafe because the buffer might contain old, sensitive data.

Comparison with Buffer.alloc

TEXT

```
// Safe but slower - fills buffer with zeros
const safeBuffer = Buffer.alloc(10); // Contains all zeros

// Unsafe but faster - may contain old data
const unsafeBuffer = Buffer.allocUnsafe(10); // May contain anything
```

Proper Usage Pattern

To safely use `allocUnsafe`, you should always follow it with a fill operation:

TEXT

```
const buf = Buffer.allocUnsafe(10);
buf.fill(0); // Now it's safe to use
```

Or use `Buffer.from()` to immediately initialize the content:

TEXT

```
const buf = Buffer.allocUnsafe(10);  
Buffer.from(buf).fill(0);
```

When to Use allocUnsafe

- Performance-critical applications where you'll immediately overwrite the buffer
- Temporary buffers that won't expose sensitive data
- Situations where you can guarantee the entire buffer will be overwritten

Why Buffer.allocUnsafe is Insecure:

When you use `allocUnsafe`, Node.js doesn't clear (zero-fill) the allocated memory block before giving it to you. This means whatever data was previously stored in that memory location remains until you overwrite it.

If you completely overwrite the entire buffer with your own data before using it, then it's considered safe to use `Buffer.allocUnsafe()`. The security risk only exists if any portion of the original uninitialized memory could potentially be exposed.

You can safely store sensitive data in an `allocUnsafe` buffer if—and only if—you completely overwrite the entire buffer before using it for sensitive operations.

When to Absolutely Use Buffer.alloc()

1. Cryptographic keys and operations
2. Authentication tokens
3. Password handling
4. Any data that would cause harm if leaked
5. Compliance scenarios (PCI DSS, HIPAA, etc.)

Remember: When dealing with sensitive data, always prefer `Buffer.alloc()` over `allocUnsafe()`. The tiny performance gain isn't worth the security risk.

CI/CD

CI/CD (Continuous Integration / Continuous Delivery or Continuous Deployment) is a set of practices that automate the integration, testing, and deployment of code to deliver software faster and more reliably.

DevOps Life Cycle

Continuous Integration (CI)

Developers integrate code changes into a shared repository multiple times a day. Each integration is automatically verified through builds and tests.

- **Build** – Compile the code and create artifacts (e.g., .jar files, .zip, bundles)
- **Test** – Run unit tests, integration tests, and static analysis to catch errors early

Benefits: early bug detection, fewer merge conflicts, more stable code.

Continuous Delivery (CD)

Code passes through an automated testing and validation pipeline, but requires human approval to deploy to production.

- **Release** – Prepare artifacts for production use
- **Staging** – Deploy to an environment that mirrors production for final testing

Continuous Deployment (CD)

Similar to Continuous Delivery, but deployment to production happens automatically if the code passes all tests — no human intervention required.

- **Deploy** – Automatically publish validated changes to the production environment

Typical CI/CD Pipeline Stages

1. **Source** – Developer pushes code to the repository (GitHub, GitLab, Bitbucket)
2. **Build** – Code is compiled and packaged as an artifact
3. **Test** – Unit, integration, functional, and security tests
4. **Staging** – Deploy to a pre-production environment for final validation
5. **Production** – Deploy to the live environment for end users

Popular Tools

- **Jenkins** – Ver [Jenkins](#)
- **GitHub Actions** – Native CI/CD integrated with GitHub
- **GitLab CI** – Built-in pipeline with self-hosted runners
- **AWS CodePipeline** – Managed CI/CD service on AWS
- **CircleCI** – Cloud-based continuous integration platform
- **Semaphore** – Fast and scalable CI/CD platform

Best Practices

- Keep the pipeline fast for immediate feedback
- Run tests in parallel when possible
- Use containers (Docker) for consistent environments
- Version pipeline configuration (Infrastructure as Code)
- Never deploy code that fails any test

Client-Server Model

The client represents an application that makes requests to the server. It could be a web or mobile application as well as any other component that interacts with the server. The client is responsible for initiating requests and processing the response. It's important to note that the term "client" is not limited to UI-based applications. In the microservices world, a client can refer to a backend application as well.

The server is the central component that receives requests from clients, processes them, and generates responses. It stores and manages resources, executes business logic, and handles data storage and retrieval. The server is responsible for processing client requests, performing necessary computations or database operations, and returning the appropriate responses.

Stateful vs Stateless architecture

Stateful

In a stateful architecture, the server maintains the state or context of a client's interactions.

- The server retains information about the client's session or past interactions, which allows it to maintain continuity across multiple requests.
- The server can store session data, authentication tokens or any other relevant information.
- This approach often requires the server to allocate resources to manage and store the client's state, which may impact scalability and resource utilization.

Stateless

In a stateless architecture, the server does not retain any information about the client's past interactions or session state.

- Each client request is self-contained and includes all the necessary information for the server to process it without relying on previous requests.
- The server treats each request independently and does not maintain any client-specific state between requests.
- Stateless architectures are typically more scalable and easier to manage as they don't require server resources for maintaining session information.
- To maintain client context on server, clients often include relevant information (such as tokens, session IDs, or authentication credentials) in each request.

It's important to note that stateless interactions do not mean that the server cannot maintain any state at all. The server can still have its internal state, such as databases or caches, to store and retrieve data. However, the client is responsible for providing all the necessary contextual information within each request, allowing the server to process it independently without relying on previous interactions.

HTTP

HTTP (Hypertext Transfer Protocol) - is a protocol used for data exchange between clients (e.g web browsers) and web servers.

Request-Response Model

HTTP request-response model is a fundamental concept which allows clients to request specific resources from servers, and servers respond with the requested data or appropriate error messages.

The request consists of:

- A method: Specifies the type of action to be performed on the server's resource (e.g., GET, POST, PUT, DELETE).
- URL (Uniform Resource Locator): Identifies the specific resource being requested.
- Headers: Include additional information about the request, such as content type, authentication credentials, caching directives, etc.
- Body (optional): Contains data sent with the request, often used in POST, PATCH or PUT requests.

The response consists of:

- Status code: Indicates the outcome of the request (e.g., 200 for "OK," 404 for "Not Found," 500 for "Internal Server Error").
 - Headers: Provide additional information about the response, such as content type, cache-control, server details, etc.
 - Body: Contains the requested resource (e.g., HTML page, JSON data) or an error message, depending on the server's processing.
-

HTTPS

HTTPS (Hypertext Transfer Protocol Secure) - is a secure version of the HTTP protocol. It adds an extra layer of security by encrypting the data exchanged between the client and the server.

- Encryption: HTTPS uses encryption algorithms to secure the communication between the client and the server. It encrypts the data being transmitted, making it unreadable to anyone who might intercept it.
- Secure Sockets Layer (SSL) or Transport Layer Security (TLS): HTTPS relies on SSL or TLS protocols to establish a secure connection. These protocols ensure that the data exchanged between the client and the server remains private and protected.
- SSL/TLS Certificates: To enable HTTPS, web servers obtain SSL/TLS certificates from trusted certificate authorities. These certificates verify the server's identity and enable secure communication. When a client connects to a website using HTTPS, it verifies the server's certificate to ensure a secure connection.

Configuration management and environments in Node.js

Configuration management

1. Using environment variables: They can be set using the command line, a .env file, or through the hosting environment. Through environment variables you can specify different configuration settings, such as database credentials or the API keys needed for third-party services.

TEXT

```
const PORT = process.env.PORT || 3000;
const DB_HOST = process.env.DB_HOST || 'localhost';
```

2. Using a configuration file: Another common way to configure Node.js apps is by using a configuration file, such as a JSON or YAML file.

TEXT

```
{
  "port": 80,
  "db": {
    "host": "example.db.host"
  }
}
```

TEXT

```
const fs = require('fs');
const config = JSON.parse(fs.readFileSync('config.json', 'utf8'));
const PORT = config.port || 3000;
const DB_HOST = config.db.host || 'localhost';
```

Graceful shutdown in Node.js

Graceful shutdown refers to the process of closing down an application in a controlled and orderly way, ensuring that all in-progress operations are completed, and no data is

lost.

If an application is shut down abruptly, it can leave resources in an inconsistent state, such as open network connections (and unhandled user requests consequently), unfinished database transactions, or data in memory that has not been persisted to disk. This can lead to data loss (which is not acceptable for production workloads), increased resource usage, and other issues. The most significant issue of not having graceful shutdown in place is that it makes bad user experience.

With a graceful shutdown, an application can be notified of the shutdown process and take appropriate steps to gracefully close down any in-progress operations, such as closing open database connections or finishing processing pending requests.

Consequently, it can prevent data loss and other issues, and ensure that resources are freed up properly.

TEXT

```
const express = require('express');
const app = express();
const server = app.listen(process.env.PORT || 3000);

const connections = [];
server.on('connection', (connection) => {
  connections.push(connection);
  connection.on('close', () => {
    connections = connections.filter((currentConnection) => currentConnection !== connection);
  });
});

function shutdown() {
  console.log('Received kill signal, shutting down gracefully');
  server.close(() => {
    console.log('Closed out remaining connections');
    process.exit(0);
  });
  setTimeout(() => {
    console.error('Could not close connections in time, forcefully shutting down');
    process.exit(1);
  }, 20000);
  connections.forEach((connection) => connection.end());
  setTimeout(() => {
    connections.forEach((connection) => connection.destroy());
  }, 10000);
}

process.on('SIGTERM', shutdown);
process.on('SIGINT', shutdown);
```

Healthcheck for Node.js apps

Implementing health checks in a Node.js application is important for ensuring that the application is functioning properly and able to handle incoming requests. Health checks provide a way to monitor the health of your application and quickly detect and respond to issues.

TEXT

```
const server = http.createServer((req, res) => {
  if (req.url === '/health') {
    res.writeHead(200);
    res.end('Healthy');
  } else {
    // Handle other requests as usual
  }
});
const PORT = process.env.PORT || 3000;
server.listen(PORT, () => {
  console.log(`Server listening on port ${PORT}`);
});
```

When the /health route is accessed, the server responds with a 200 OK status code and a simple text message indicating that the server is healthy.

TEXT

```
const express = require('express');
const mysql = require('mysql2');
const port = process.env.PORT || 3000;
const dbConfig = {
  host: process.env.DB_HOST || 'localhost',
  user: process.env.DB_USER || 'root',
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME || 'mydatabase',
};
const app = express();
const pool = mysql.createPool(dbConfig);

app.get('/health', (req, res) => {
  pool.getConnection((err, connection) => {
    if (err) {
      return res.status(500).send('Error connecting to database');
    }
    res.status(200).send('OK');
    connection.release();
  });
});
app.listen(port, () => console.log(`Server listening on port ${port}`));
```

Here's an example response when the health check is successful:

TEXT

```
{
  "status": "ok",
  "info": {
    "database": {
      "status": "up"
    }
  }
}
```

And here's an example response when the health check fails:

TEXT

```
{
  "status": "error",
  "errors": {
    "database": {
      "status": "down",
      "message": "Database connection timed out after 300ms"
    }
  }
}
```

Console

Node.js provides a console module which provides tons of very useful ways to interact with the command line.

Basic Output

The most basic and most used method is `console.log()`, which prints the string you pass to it to the console.

TEXT

```
const name = 'Bruce';
const surname = 'Wayne';
console.log(name, surname); // Bruce Wayne
```

Clear the console

TEXT

```
console.clear();
```

Counting elements

TEXT

```
const oranges = ['orange', 'orange'];
const apples = ['just one apple'];
oranges.forEach(fruit => {
  console.count(fruit);
});
apples.forEach(fruit => {
  console.count(fruit);
});
// The output:
// orange: 1
// orange: 2
// just one apple: 1
```

Calculate the time spent

TEXT

```
const doSomething = () => console.log('test');
const measureDoingSomething = () => {
  console.time('doSomething()');
  doSomething();
  console.timeEnd('doSomething()');
};
measureDoingSomething();
// The output:
// test
// doSomething(): 0.065673828125 ms
```

Using table

TEXT

```
const authorsAndBooks = [
  ['Tolkien', 'Lord of The Rings'],
  ['Rutger', 'Utopia For Realists'],
  ['Sinek', 'Leaders Eat Last'],
  ['Eyal', 'Habit'],
];
console.table(authorsAndBooks);
```

TEXT

```
const inventory = { apples: 200, mangoes: 50, avocados: 300, kiwis: 50 };
console.table(inventory);
```

Print errors and warnings

- `console.error` writes an error message to the console:

TEXT

```
console.error('error');
```

- `console.warn` method writes a warning to the console:

TEXT

```
console.warn('warn');
```

Style the output

You can style the browser output of your message by using %c directive to apply a CSS style to console output.

TEXT

```
console.log("This is %cMy stylish message", "color: lime; font-style: italic");
```

Sadly, the same thing doesn't work for Node.js, exactly. Since node is running in a terminal, it's not running in a browser that understands CSS. Instead, we can change the colors of the text through some odd-looking characters: \x1b[31m makes the text red.

TEXT

```
console.log('\x1b[31mAnd now I\'m red!');
```

Errors

An Error in Node.js is any instance of the Error object.

- **ReferenceError** - when a reference made to a variable/item is broken, which means the variable/item doesn't exist.
- **RangeError** - when a number is outside an allowable range of values.
- **TypeError** - when an operation is performed on a wrong data type
- **URIError** - indicates that one of the global URI handling functions was used in an incompatible way with its definition.
- **EvalError** - is used to identify errors when using the global eval() function.
- **SyntaxError** - when we type code that the JS engine can't understand.

User-defined errors

TEXT

```
const error = new Error("I am a teapot!");
error.code = 418;
console.log(error.name); // Error
console.log(error.message); // I am a teapot!
console.log(error.stack);
/* Error: I am a teapot!
   at Object.<anonymous> (/Users/anastasia/Desktop/node/app.js:1:15)
   <truncated for brevity>
   at node:internal/main/run_main_module:17:47 */
```

Why Do You Need Error Handling

Error handling is a way to find bugs and solve them as quickly as humanly possible and helps execution to resume when interrupted. It centralizes all errors and lets you enable alerting and notifications so you know when and how your code breaks.

Operational Errors

These errors are expected in the Node.js runtime and should be dealt with in a proper way. This does not mean the application itself has bugs. It means they need to be handled properly.

- failed to connect to server
- a database connection is lost
- invalid user input
- system is out of memory

Programmer errors

Programmer errors are mistakes in the logic or syntax of the program that can only be corrected by changing the source code.

- syntax errors, such as failing to close a curly brace
- bad parameters when calling a function
- reference errors when you misspell a variable, function, or property name
- unresolved promise
- failing to handle an operational error

How to deliver errors

Throw exception

In Node.js (and JavaScript in general), when an error is thrown using the `throw` statement, it creates an exception that propagates up through the function call stack until it is caught by a `try/catch` block. If it's not caught, it will crash the program.

TEXT

```
function square(num) {  
  if (typeof num === 'number') {  
    throw new TypeError(`Expected number but got: ${typeof num}`);  
  }  
  return num * num;  
}  
  
try {  
  square('8');  
} catch (err) {  
  console.log(err.message); // Expected number but got: string  
}
```

Databases. NoSQL

NoSQL does not store data in a relational format. The data does not need to have a strict schema nor the usual SQL table structure.

Key Features of NoSQL:

- Flexible Schema – No fixed structure; supports dynamic data.
- High Scalability – Easily scales horizontally across servers.
- Fast Performance – Optimized for quick reads/writes.
- Multiple Data Models – Supports document, key-value, column, and graph types.

ACID

A – Atomicity Transactions are all-or-nothing: either everything succeeds, or nothing happens.

C – Consistency The database remains in a valid state before and after the transaction.

I – Isolation Concurrent transactions do not interfere with each other.

D – Durability Once a transaction is committed, it is permanently saved—even if the system crashes.

If ACID properties belong to RDBMSs, BASE properties belong to NoSQL databases.

BASE

- Basically Available: Writing and reading operations are available as much as possible even if it's a failure. Will ensure availability of data by spreading and replicating it across the nodes of the database cluster.
- Soft State: No guarantees that the data eventually becomes consistent.
- Eventual Consistency The components in the system may not reflect the same value/state of a record at a given point in time. They will settle it eventually.

CAP

The CAP theorem states that it's impossible for a database system to achieve Consistency, Availability, and Partition Tolerance at the same time.

- Consistency: Data should be sequential consistent. Refers to linearizability across all nodes of a distributed system. All nodes in a system return same data after a successful write operation. Consistency is definitely an important characteristic of any distributed NoSQL database. However, not all of them can provide it.

- Availability: Every request served by a non-failing (Fault Tolerance) node must result in a response in a reasonable amount of time.
- Partition tolerance: In the case of partition failure, system will continue to work and provide consistent data.

It's theoretically impossible to have all 3 requirements met, so a combination of 2 must be chosen and this is usually the deciding factor in what technology is used.

- AP systems. Highly available and partitioning tolerant databases that will always respond to your requests, no matter what happens.
- CP systems. Consistent and partitioning tolerant databases that will care for the data no matter what.
- AC systems. Highly available and consistent databases will always respond to all your requests from any node with the latest version of the data.

CAP Category

CAP Category	DB Types	DB
AP	Key-Value Pair	Riak
CP	Key-Value Pair	Redis
AP	Document-oriented	CouchBase
CP	Document-oriented	MongoDB
AP	Column-oriented	Cassandra
CP	Column-oriented	HBase
CA	Graph	Neo4J

SQL VS NoSQL

	SQL	NoSQL
Definition	SQL databases are primarily called RDBMS or Relational Databases	NoSQL databases are primarily called as Non-relational or distributed database
Query Language	Structured query language (SQL)	No query language
Type	SQL databases are table based databases	NoSQL databases can be document based, key-value pairs, graph databases
Schema	SQL databases have a predefined schema	NoSQL databases use dynamic schema for unstructured data
Scalability	SQL databases are vertically scalable	NoSQL databases are horizontally scalable
Performance	Need a larger server to accommodate the increasing amount of data	Can add new servers to what you already have

Types of NoSQL

Key-value DB

It is the least complicated type of NoSQL databases that using dictionary-like storing method and doesn't require a predefined schema.

Key-Value DBs provide a way to store and access data using simple operations:

- Get (key) - Retrieve value by provided key
- Put (key, value) - Add new key-value pair
- Update (key, value) - Update value by provided key
- Delete (key) - delete key-value pair

Specifics

- Consistency - applicable for operations on a single key in a key-value store
- Query - can use query by the key only
- Scaling - scale by a process called sharding

Examples:

- Redis
- Amazon DynamoDB

Document-oriented DB

This database is designed to store and query data in the form of documents, usually JSON, but also XML or YAML. Objects within a database are stored completely within one document.

TEXT

```
{
  "ID": "001",
  "Book": "NodeJS Cookbook",
  "Genre": "Work and Education",
  "Author": "Bethany Griggs"
}
```

Specifics

- Consistency - some databases offers eventual consistency with a period of inconsistency
- Query - can use query based on unique identifiers or field values
- Scaling - databases are distributed and resilient. They allow for horizontal scaling

Examples

- MongoDB

Graphs

A graph database is a database designed for storing and querying data that is connected via defined relationships. Graphs contain nodes, edges, and properties. Data points in a graph database are nodes and these nodes are connected to related data via edges. The data attached to each node are known as properties. Graph databases aren't restricted by predefined schema. There are two types of graphs: property graphs and RDF graphs. The property graph focuses on analytics and querying, while the RDF graph emphasizes data integration. Both types of graph consist of a collection of points (vertices) and the connections between those points (edges).

Specifics

- Consistency - offers eventual consistency with a period of inconsistency
- Query - can use specially defined query language like Cypher, SPARQL
- Scaling - databases are distributed and resilient. They allow for horizontal scaling

Examples

- Neo4j
- DataStax Astra DB

Column-oriented Graph

These databases are designed to speed up workloads that operate on column data across all records/rows. They organize data by field and keeping all of the data associated with a field next to each other in memory storage.

Specifics

- Consistency - offers eventual consistency with a period of inconsistency
- Query - can use traditional SQL to perform queries
- Scaling - databases are distribute and can be sharded or partitioned across multiple servers
- Compression - databases are highly compressed compared to relational databases. Easy to optimize storage size.

Examples

- Amazon Redshift

Conceptual techniques

Denormalization: could be used to simplify and optimize query processing by grouping all the data to query in one place. Can be defined as the copying of the same data into multiple tables or documents. Drawback is the data volume increases for various parameters, considerably increasing the data volume.

Databases:

- Key-Value Stores
- Document Databases

Aggregates: could be used to create nested entities with complex internal structures thanks to the soft schema. It helps to reduce joins by minimizing one-to-one relationships.

Databases:

- Key-Value Stores
- Document Databases

Application Side Joins: could be used during designing and developing the application. Compared to relational databases - queries are performed at query execution time. This tends to increase a performance penalty.

Databases:

- Key-Value Stores
- Document Databases
- Graph Databases

General modeling techniques

Atomic Aggregates. Because Aggregates allow to store a single entity as one document, row or key-value pair, it is possible to update this entity atomically.

Enumerable keys. Because entity can be distributed across multiple servers, it is possible to use one hashed key. Drawback is sorting such keys may add a bit of complexity and a performance hit.

Index table. This technique requires to create an additional table. The keys in this table follow the access pattern and simplify query execution. Working with this technique could result an additional performance penalty and become a consistency issue.

Composite key index. This technique allows to create composite key to simplify query execution and aggregate data by a partial key. Composite keys are great when using ordered keys.

Inverted Lookup. This technique allows to use an index to find data that meets a criteria, but aggregate data using original representation or full scans. Using such indexes it is possible to intersect or unify corresponding data.

Object Document Mapper

The Object Document Mapper (ODM) is an interface that treats a document as a tree structure. Similar to Object Relational Mapping (ORM), it is used to change the structure and content of a document. And also allows to define data schemas which represent the data structure. The usage of ODMs additionally simplifies the process of changing the database engine whenever the need arises.

Multiple ODMs are designed to support different NoSQL databases:

- Express-Cassandra is designed to work with Node.js for a NoSQL database like Cassandra
- Mongoose is designed for MongoDB database
- Amazon DynamoDB DataMapper is a helper for DynamoDB

Patrones de Diseño / Design Patterns

Factory functions

Las factory functions en JavaScript son funciones que crean y devuelven objetos. Son una alternativa a las clases y a los constructores para crear múltiples instancias de objetos con las mismas propiedades y métodos.

Se llaman "factory" porque actúan como una fábrica que genera objetos. A diferencia de usar clases, con las factory functions no se necesita la palabra clave `new` para crear instancias.

TEXT

```
function createUser(name, age) {
  return {
    name,
    age,
    greet() {
      return `Hello, my name is ${this.name} and I am ${this.age} years old`;
    },
  };
}

const user1 = createUser('Alice', 30);
const user2 = createUser('Bob', 25);

console.log(user1.greet());
// Hello, my name is Alice and I am 30 years old.
console.log(user2.greet());
// Hello, my name is Bob and I am 25 years old.
```

Adapter method

Adapter Pattern in JavaScript is a structural design pattern that allows you to make one interface or object work with another that has a different interface.

Example

TEXT

```
class Calculator {
  operation(num1, num2, operation) {
    switch (operation) {
      case 'multiplication':
        return num1 * num2;
      case 'division':
        return num1 / num2;
      default:
        return NaN;
    }
  }
}

export default Calculator;
```

and we can use the class like this:

TEXT

```
import Calculator from "./Calculator"
const calculator = new Calculator();
const multi = calculator.operation(1, 2, 'multiplication');
console.log(multi); //output 2
const div = calculator.operation(10, 2, 'division');
console.log(div); //output 5
```

The time passes and the project gets bigger and bigger. It's time for a complete refactor of the Calculator class!

TEXT

```
class Calculator {
  add(num1, num2) {
    return num1 + num2;
  }
  div(num1, num2) {
    return num1 / num2;
  }
  mult(num1, num2) {
    return num1 * num2;
  }
}

export default Calculator;
```

This is the new way of using the calculator:

TEXT

```
import Calculator from "./Calculator"
const calculator = new Calculator();
const sum = calculator.add(1, 2);
console.log(sum); //output 3
const sub = calculator.div(10, 2);
console.log(sub); //output 5
```

As we can see, the refactor is no more compatible with the last version. For this we need to create an adaptor to make it work with both versions.

TEXT

```
import Calculator from "./Calculator";
class CalculatorAdapter {
  constructor() {
    this.calculator = new Calculator();
  }
  operation(num1, num2, operation) {
    switch (operation) {
      case "add":
        return this.calculator.add(num1, num2);
      case "multiplication":
        return this.calculator.mult(num1, num2);
      case "division":
        return this.calculator.div(num1, num2);
      default:
        return NaN;
    }
  }
}
export default CalculatorAdapter;
```

Now we can use the the original Calculator with the Adapter.

TEXT

```
import Calculator from "./Calculator";
import CalculatorAdapter from "./CalculatorAdapter";

const calcAdapter = new CalculatorAdapter();
const sumAdapter = calcAdapter.operation(2, 2, "multiplication");
console.log(sumAdapter); //output 4

const calculator = new Calculator();
const sum = calculator.mult(2, 2);
console.log(sum); //output 4
```

Docker

¿Qué es Docker?

Docker es una plataforma que permite empaquetar una aplicación con todas sus dependencias en una unidad estandarizada llamada **contenedor**. El problema clásico que resuelve es el famoso "en mi máquina sí funciona": al contenerizar la aplicación, esta se comporta igual sin importar dónde se ejecute (local, servidor, nube).

A diferencia de una máquina virtual, un contenedor no incluye un sistema operativo completo. Comparte el kernel del host, lo que lo hace mucho más liviano y rápido de iniciar.

Dockerfile

¿Qué es un Dockerfile?

Un Dockerfile es un archivo de texto que contiene las instrucciones para construir una **imagen** de Docker. Esa imagen es una plantilla reutilizable que incluye todo lo necesario para ejecutar una aplicación: sistema operativo base, dependencias, librerías, archivos de código y comandos de inicio.

A continuación se explica cada instrucción de un Dockerfile típico para una aplicación Node.js.

FROM — Imagen base

DOCKERFILE

```
FROM node:22-alpine
```

Toda imagen parte de una **imagen base**. En este caso usamos `node:22-alpine`, una imagen oficial de Node.js versión 22. El sufijo `alpine` indica que utiliza Alpine Linux, una distribución extremadamente liviana que mantiene la imagen pequeña y eficiente. Esto se traduce en builds más rápidos y menos espacio en disco.

WORKDIR — Directorio de trabajo

DOCKERFILE

```
WORKDIR /app
```

Define el **directorio de trabajo** dentro del contenedor. Si la carpeta no existe, Docker la crea automáticamente. Todas las instrucciones posteriores (`COPY` , `RUN` , `CMD`) se ejecutarán dentro de esta carpeta. No es necesario que tu proyecto local tenga una carpeta llamada `app` ; esto es interno del contenedor.

COPY — Copiar archivos al contenedor

DOCKERFILE

```
COPY package*.json .
```

Copia `package.json` y `package-lock.json` del host al directorio de trabajo del contenedor. El `.` al final significa "aquí, en el WORKDIR actual". Es buena práctica copiar primero solo los archivos de dependencias para aprovechar el **sistema de capas** de Docker: si no cambian, esta capa se reutiliza de la caché en builds posteriores.

RUN — Ejecutar comandos durante el build

DOCKERFILE

```
RUN npm install
```

Ejecuta un comando dentro del contenedor durante la construcción de la imagen. En este caso instala todas las dependencias definidas en `package.json` . El `node_modules` resultante queda dentro del contenedor, aislado del sistema host.

COPY — Copiar el resto del proyecto

DOCKERFILE

```
COPY . .
```

Copia el resto de los archivos del proyecto (código fuente, rutas, controladores, etc.) al contenedor. Se hace después de instalar dependencias para que los cambios en el código no invaliden la caché de `npm install` .

EXPOSE — Exponer puertos

DOCKERFILE

```
EXPOSE 3000
```

Documenta que la aplicación usará el puerto 3000. Es una declaración informativa: no publica el puerto por sí sola. Sirve como documentación para quien use la imagen y como referencia para el mapeo de puertos en tiempo de ejecución.

La publicación real del puerto se hace al ejecutar el contenedor con `-p`.

CMD — Comando de inicio

DOCKERFILE

```
CMD ["npm", "start"]
```

Define el comando que se ejecutará cuando el contenedor se inicie. Cada palabra del comando va como un elemento separado del arreglo (formato exec). También se podría escribir como `CMD npm start` (formato shell), pero el formato exec es más predecible y seguro.

Dockerfile completo

DOCKERFILE

```
FROM node:22-alpine

WORKDIR /app

COPY package*.json .

RUN npm install

COPY . .

EXPOSE 3000

CMD ["npm", "start"]
```

Construir la imagen

docker build

`docker build` procesa el Dockerfile y genera una imagen. Lee las instrucciones línea por línea y crea una **capa** por cada instrucción, lo que permite cachear y reutilizar capas inmutables en builds futuros.

BASH

```
docker build -t mi-api .
```

Desglose del comando:

- `docker build` — construye la imagen a partir de un Dockerfile.

- `-t mi-api` — asigna un nombre (tag) a la imagen. Sin tag, la imagen se identifica solo por un ID.
- `.` — el **contexto de construcción**. Docker empaqueta todo el contenido del directorio actual y lo envía al demonio para construir la imagen. Por eso es importante tener un `.dockerignore` para excluir archivos innecesarios.

Caché de capas

Cada instrucción del Dockerfile genera una capa. Docker cachea cada capa y si una instrucción no ha cambiado entre builds, reutiliza la capa cacheada. Por eso se copia `package*.json` antes que el resto del código: si solo cambias el código fuente, `npm install` no se vuelve a ejecutar.

Ver las imágenes disponibles

BASH

```
docker images
```

TEXT

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
mi-api	latest	a1b2c3d4e5f6	2 minutes ago	180MB

`docker images` lista todas las imágenes almacenadas localmente, incluyendo las que construiste y las que descargaste de registries como Docker Hub.

Imagen vs Contenedor

	Imagen	Contenedor
Qué es	Plantilla o molde	Instancia en ejecución
Analogía	Receta de cocina	Plato servido
Estado	Inmutable	Modificable efímero
Persistencia	Permanente	Temporal (se pierde al eliminar)

La **imagen** contiene todo lo necesario: sistema base, dependencias, código y configuración. Es estática: una vez creada no cambia.

El **contenedor** es una instancia en ejecución creada a partir de una imagen. Tiene su propia memoria, procesos y sistema de archivos. Puedes crear múltiples contenedores desde una misma imagen.

Flujo de trabajo

1. Escribes el **Dockerfile** con las instrucciones.
2. Ejecutas `docker build` → obtienes una **imagen**.
3. Ejecutas `docker run` → se crea y ejecuta un **contenedor** a partir de esa imagen.

Si modificas el código fuente, la imagen no se actualiza sola. Debes repetir el ciclo: `docker build` para generar una nueva imagen, y `docker run` para crear un nuevo contenedor.

Ejecutar un contenedor

docker run

`docker run` crea un contenedor a partir de una imagen y lo ejecuta.

BASH

```
docker run mi-api
```

Lo que Docker hace internamente:

1. Busca la imagen `mi-api` localmente. Si no existe, la descarga de Docker Hub.
2. Crea un contenedor nuevo a partir de esa imagen.
3. Ejecuta el comando definido en `CMD` (ej. `npm start`).
4. La aplicación empieza a correr dentro del contenedor.

Principales flags

`-d` (detached)

BASH

```
docker run -d mi-api
```

Ejecuta el contenedor en **segundo plano**. Sin `-d`, los logs se muestran en la terminal y esta queda ocupada. Con `-d`, recuperas el control inmediatamente.

`-p` (port mapping)

BASH

```
docker run -p 3000:3000 mi-api
```

Formato: `-p <puerto_host>:<puerto_contenedor>`

Conecta un puerto de tu computadora al puerto del contenedor. El contenedor tiene su propia red interna y no es accesible directamente desde el host. El mapeo crea un puente:

- `-p 3000:3000` → accedes desde `http://localhost:3000`
- `-p 8080:3000` → el contenedor sigue usando el 3000, pero accedes desde `http://localhost:8080`

`--name`

BASH

```
docker run --name api mi-api
```

Asigna un nombre personalizado al contenedor. Si no se asigna, Docker genera uno aleatorio como `happy_einstein`. El nombre facilita administrarlo (logs, stop, rm).

`--rm`

BASH

```
docker run --rm -p 5002:3000 02-node-web
```

Elimina el contenedor automáticamente cuando se detiene. Ideal para pruebas rápidas donde no necesitas conservar el contenedor después de usarlo.

`-e` (environment variables)

BASH

```
docker run -d -p 5005:3000 -e SALUDO="Hola" --name web-env 02-node-web
```

Inyecta **variables de entorno** al contenedor. Útil para pasar configuraciones como API keys, URLs de base de datos o entornos (`NODE_ENV`).

`--restart`

BASH

```
docker run -d --restart always --name api mi-api
```

Define la política de reinicio. Opciones comunes:

- `no` — no reiniciar (default).
- `always` — reiniciar siempre que se detenga.
- `on-failure` — reiniciar solo si termina con error.
- `unless-stopped` — reiniciar a menos que lo detengas explícitamente.

Comando completo típico

BASH

```
docker run -d -p 3000:3000 --name api mi-api
```

Significa: "Crea un contenedor llamado `api`, usando la imagen `mi-api`, ejecútalo en segundo plano y conecta el puerto 3000 del contenedor con el puerto 3000 de mi computadora."

Comandos esenciales

docker ps

BASH

```
docker ps
```

Lista los contenedores **en ejecución**. Muestra ID, nombre, imagen, estado, puertos mapeados y tiempo activo.

BASH

```
docker ps -a
```

Lista **todos** los contenedores, incluyendo los detenidos. Útil para encontrar contenedores que ya terminaron pero siguen ocupando espacio.

BASH

```
docker ps -q
```

Muestra solo los IDs numéricos. Útil para usar en combinación con otros comandos (ej. `docker stop $(docker ps -q)`).

docker stop / start

BASH

```
docker stop <id_o_nombre>
```

Detiene un contenedor de forma controlada (envía SIGTERM y espera unos segundos antes de forzar el cierre).

BASH

```
docker start <id_o_nombre>
```

Inicia un contenedor que estaba detenido. Los datos dentro del contenedor se conservan.

docker rm

BASH

```
docker rm <id_o_nombre>
```

Elimina un contenedor. Debe estar detenido primero.

BASH

```
docker rm -f <id_o_nombre>
```

Fuerza la eliminación incluso si el contenedor está en ejecución.

BASH

```
docker container prune
```

Elimina todos los contenedores detenidos de una sola vez.

docker logs

BASH

```
docker logs <id_o_nombre>
```

Muestra los logs (salida estándar y de error) de un contenedor.

BASH

```
docker logs -f <id_o_nombre>
```

Muestra los logs en **tiempo real** (follow), similar a `tail -f`. Útil para debug mientras la app está corriendo.

BASH

```
docker logs --tail 50 <id_o_nombre>
```

Muestra solo las últimas 50 líneas.

docker exec

BASH

```
docker exec -it <id_o_nombre> sh
```

Ejecuta un comando **dentro de un contenedor que ya está corriendo**. Es como abrir una terminal dentro del contenedor.

- `-i` — interactivo (mantiene STDIN abierto).
- `-t` — asigna un pseudo-TTY (terminal).
- `sh` — abre una shell. También funciona `bash` si la imagen lo tiene.

Útil para inspeccionar archivos, variables de entorno o procesos dentro del contenedor.

BASH

```
docker exec <id> cat /app/.env  
docker exec <id> ls -la
```

docker images

BASH

```
docker images
```

Lista las imágenes disponibles localmente (las que construiste y las descargadas).

BASH

```
docker rmi <imagen_id>
```

Elimina una imagen. Si hay contenedores que la usan, debes eliminarlos primero.

docker pull / push

BASH

```
docker pull node:22-alpine
```

Descarga una imagen de Docker Hub (o cualquier registry configurado) sin ejecutarla.

BASH

```
docker push usuario/mi-api
```

Sube una imagen a un registry (Docker Hub, AWS ECR, etc.). La imagen debe estar etiquetada con el nombre del registry.

docker inspect

BASH

```
docker inspect <id_o_nombre>
```

Muestra información detallada de un contenedor o imagen en formato JSON: configuración, red, variables de entorno, montajes, etc.

docker system

BASH

```
docker system df
```

Muestra el uso de espacio en disco de Docker: imágenes, contenedores, volúmenes y build cache. Útil para diagnosticar por qué Docker ocupa tanto espacio.

TEXT

TYPE	TOTAL	ACTIVE	SIZE	RECLAIMABLE
Images	5	2	2.1GB	1.3GB (62%)
Containers	8	1	50MB	45MB (90%)
Local Volumes	3	1	500MB	200MB (40%)
Build Cache	-	-	300MB	300MB

BASH

```
docker system prune
```

Elimina contenedores detenidos, redes no usadas, imágenes colgantes y build cache.

BASH

```
docker system prune -a
```

Más agresivo: también elimina imágenes **no usadas** (no solo las colgantes). Libera la mayor cantidad de espacio posible.

.dockerignore

Funciona como `.gitignore` : excluye archivos del **contexto de build** para que no se envíen al demonio de Docker. Esto acelera el build y reduce el tamaño de la imagen final.

DOCKERIGNORE

```
node_modules
.git
.env
.env.local
*.log
dist
.vscode
.DS_Store
```

Cada línea es un patrón de archivo o carpeta que Docker ignorará al construir.

Volúmenes

Los contenedores son efímeros: cuando se eliminan, los datos que contienen se pierden. Los **volúmenes** permiten persistir datos fuera del ciclo de vida del contenedor.

Tipos de montajes

Named volumes — administrados por Docker:

BASH

```
docker run -v mi-volumen:/app/data mi-api
```

Docker crea y gestiona el volumen. Los datos persisten incluso si eliminas el contenedor.

Bind mounts — carpeta del host montada directamente:

BASH

```
docker run -v $(pwd):/app mi-api
```

Monta el directorio actual del host en `/app` del contenedor. Útil para desarrollo porque los cambios locales se reflejan al instante.

tmpfs mounts — en memoria (solo Linux):

BASH

```
docker run --tmpfs /app/temp mi-api
```

Datos temporales que se pierden al detener el contenedor.

docker volume

BASH

```
docker volume ls
docker volume create mi-volumen
docker volume rm mi-volumen
docker volume prune
```

Redes

Cada contenedor tiene su propia interfaz de red. Docker ofrece varios **drivers de red** para controlar cómo se comunican los contenedores entre sí y con el exterior.

Drivers

bridge (default) Crea una red privada interna. Los contenedores en la misma red bridge pueden comunicarse entre sí por nombre. Aislados de contenedores en otras bridges.

BASH

```
docker network create mi-red
docker run -d --net mi-red --name api mi-api
docker run -d --net mi-red --name db postgres
# api puede conectarse a db usando el nombre "db"
```

host El contenedor usa la red del host directamente. No hay aislamiento de red. Útil cuando necesitas máximo rendimiento de red.

none Sin red. Contenedor completamente aislado.

Comandos de red

BASH

<code>docker network ls</code>	<code># listar redes</code>
<code>docker network create mi-red</code>	<code># crear red personalizada</code>
<code>docker network connect mi-red api</code>	<code># conectar contenedor a una red</code>
<code>docker network inspect mi-red</code>	<code># ver detalles</code>
<code>docker network prune</code>	<code># eliminar redes no usadas</code>

Docker Compose

Cuando una aplicación tiene múltiples servicios (API, base de datos, caché, cola de mensajes), manejarlos individualmente con `docker run` se vuelve tedioso. **Docker Compose** permite definir y ejecutar aplicaciones multi-contenedor con un solo archivo YAML.

docker-compose.yml

YAML

```
services:
  api:
    build: .
    ports:
      - "3000:3000"
    environment:
      - NODE_ENV=production
      - DB_HOST=db
    depends_on:
      - db

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: mi-app
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

Comandos de Compose

BASH

```
docker compose up -d      # levantar todos los servicios
docker compose down       # detener y eliminar contenedores/redes
docker compose down -v    # lo mismo + elimina volúmenes
docker compose logs -f    # logs de todos los servicios
docker compose logs -f api # logs de un servicio específico
docker compose ps         # estado de los servicios
docker compose exec api sh # ejecutar comando en un servicio
docker compose build       # reconstruir imágenes
docker compose up -d --build # reconstruir y levantar
```

Extensiones de VSCode

- **Docker** (Microsoft) — administrar contenedores, imágenes, Dockerfiles y Compose directamente desde el IDE.
- **Dev Containers** (Microsoft) — desarrollar dentro de contenedores usando VSCode remoto.

Docker Hub

Docker Hub es el registry público por defecto de Docker. Almacena y distribuye imágenes de contenedores.

Si ejecutas `docker run` o `docker pull` con una imagen que no existe localmente, Docker la busca automáticamente en Docker Hub y la descarga.

BASH

```
docker pull node:22-alpine      # descargar imagen oficial
docker pull nginx:latest        # descargar Nginx
docker tag mi-api usuario/mi-api:v1 # etiquetar para subir
docker push usuario/mi-api:v1    # subir a Docker Hub
```

Las imágenes se versionan con **tags**. `node:22-alpine` significa: imagen `node`, tag `22-alpine` (versión 22 con Alpine). `latest` es el tag por defecto si no se especifica uno.

Licenciamiento

Docker Desktop solía ser gratuito para todos, pero desde agosto de 2021 cambió su modelo de licencia:

- **Docker Desktop** requiere suscripción paga para uso empresarial (empresas con más de 250 empleados o ingresos superiores a \$10M USD).
- **Docker Engine** (el motor de línea de comandos) sigue siendo open source y gratuito bajo la licencia Apache 2.0.
- El uso personal, educativo y en proyectos open source sigue siendo gratuito en Docker Desktop.

Opciones gratuitas

- Usar solo Docker Engine (CLI) sin Docker Desktop.
- Usar alternativas open source como Podman o Colima.

Alternativas Open Source

Podman

Podman es un motor de contenedores open source, diseñado como **drop-in replacement** de Docker. Desarrollado por Red Hat.

- Soporta Dockerfile y puede ejecutar imágenes de Docker sin conversión.
- **No requiere un daemon central:** cada contenedor se ejecuta como un proceso hijo directo, lo que mejora seguridad y simplicidad.
- **No necesita privilegios de root** para ejecutar contenedores.
- CLI casi idéntica a Docker (puedes usar `alias docker=podman`).
- Compatible con Docker Compose mediante `podman-compose` .

BASH

```
podman build -t mi-api .
podman run -d -p 3000:3000 mi-api
podman ps
```

Colima

Colima es una alternativa ligera a Docker Desktop para macOS. Ejecuta contenedores usando Lima (máquinas virtuales) bajo el capó.

- No requiere Docker Desktop.
- Open source y gratuito.
- Compatible con la CLI de Docker.
- Soporta arquitectura ARM e Intel.

BASH

```
colima start      # inicia la VM
colima stop       # detiene la VM
```

Otras alternativas

- **containerd** — runtime de contenedores usado internamente por Docker, también se puede usar directamente.
- **nerdctl** — CLI compatible con Docker para containerd.
- **Rancher Desktop** — alternativa multiplataforma que usa containerd y Kubernetes.

Docker

Think of Docker as a suitcase. Just as a suitcase allows you to pack your clothes and essentials for any journey, Docker lets you package your application and all its dependencies for easy transportation. This 'suitcase' ensures your application works seamlessly, no matter where it is deployed - your computer, a colleague's machine, or a server in the cloud.

Docker enables developers to package an application with all its dependencies into a standardized unit called a Docker container. This container ensures the application behaves the same way, irrespective of where it's deployed, thereby resolving environment inconsistencies.

A Docker Image is the skeleton of this container. It's a lightweight, standalone, executable package containing everything needed to run an application - the code, runtime, libraries, and environment variables. Docker creates these images using a Dockerfile, a file containing all the commands to assemble the Docker Image. Essentially, a Dockerfile serves as a set of instructions for Docker to build an image.

Docker Compose

As your application becomes more complex with multiple components that need to interact, managing these becomes more challenging. That's where Docker Compose comes in, acting like an organizer. It lets you outline the interactions of each component of your application and handle them all together in a simpler, more coordinated way.

Together, Docker and Docker Compose make managing and deploying complex applications easier, smoother and more reliable.

Docker licensing model

Docker has several key components that allow it to build, transport, and manage containers efficiently. Here are the fundamental parts:

- **Docker Engine:** The heart of Docker, it's a lightweight runtime and tooling that builds and manages containers.
- **Docker CLI (Command Line Interface):** A tool that allows users to interact with Docker directly using commands. This is where user inputs commands to interact with the Docker daemon.
- **Docker Desktop (UI):** An application that is designed to facilitate the developer's workflow by providing an intuitive user interface to interact with Docker.

Podman & Podman Compose

It is an open-source, free to use tool that is designed to be a drop-in replacement for Docker. Podman supports Dockerfile syntax, can pull and run Docker images. Podman can be used instead of Docker without having to modify existing Dockerfiles or images.

podman-compose is a command-line tool for defining and running multi-container Docker applications with Podman. It is similar to Docker Compose, but instead of requiring Docker, it uses Podman container engine to build and run the containers.

Node.js Event Loop

The Event Loop is what allows Node.js to perform non-blocking I/O operations despite being single-threaded.

Event Loop plays an orchestration role for queues. Its key responsibility is to iterate over a set of queues and execute callbacks registered for those events sequentially.

There are 6 Event Loop phases:

1. Timers Phase

Executes callbacks scheduled by `setTimeout()` and `setInterval()`. These callbacks are not guaranteed to run exactly at the time you set — they run as soon as possible after the time has passed, depending on other phases.

TEXT

```
setTimeout(() => {  
  console.log('Timer callback');  
}, 1000);
```

If a timer is set for 100ms, but the poll phase takes 150ms, it runs immediately in the next timers phase.

2. Pending Callbacks

Pending callbacks — I/O-related callbacks will be executed, delayed to the next loop iteration. For example, if you registered a callback fired after write or read to file has finished, or some network operation is completed, this callback will run in this phase (I/O Events queue).

3. Idle, Prepare

Used internally by Node.js and libuv (you won't directly interact with this). It prepares for the next poll phase.

4. Poll Phase

This is where most of the I/O happens! Event Loop looks for new I/O events and does its best to execute their callbacks immediately if possible. If not, it will defer a callback execution and register this as a pending callback (I/O Events queue).

What happens here:

- If there are I/O events ready (like file read complete, new data on socket), their callbacks are executed here.
- If no timers are ready, and no immediate callbacks, the event loop may block here and wait for I/O.

5. Check Phase

Executes callbacks passed to `setImmediate()`.

TEXT

```
setImmediate(() => {  
    console.log('Check phase - setImmediate callback');  
});
```

Libuv

libuv is a C-based library that Node.js uses under the hood to handle asynchronous operations. It provides:

- Asynchronous I/O (File System, Networking, Timers) What it is: A programming model where operations (like reading files or network requests) happen in the background without blocking the main thread.
- Event Loop Implementation What it is: The core mechanism that processes asynchronous callbacks in Node.js.
- Thread Pool for Blocking Operations What it is: A set of worker threads that handle operations the OS can't do asynchronously.

Microtasks vs Macrotasks

Microtasks (higher priority):

Microtasks are smaller, high-priority tasks that run right after the current task finishes, before the event loop moves to the next macro task.

- Smaller, higher-priority tasks executed between Event Loop phases
- Must complete before the Event Loop continues

TEXT

```
console.log('script start');
setTimeout(() => {
  console.log('setTimeout'); // macro
}, 0);
Promise.resolve().then(() => {
  console.log('promise1'); // micro
});
queueMicrotask(() => {
  console.log('microtask'); // micro
});
console.log('script end');
```

output

TEXT

```
script start
script end
promise1
microtask
setTimeout
```

1. Synchronous code runs first.
2. Then microtasks: promise1 and microtask.
3. Then macro task: setTimeout.

Macro Tasks (a.k.a. "Tasks")

These are the main units of work that the JavaScript event loop pulls from the task queue.

- Larger, heavier operations processed in the Event Loop phases
- Each macrotask runs in a new Event Loop cycle

Express and Layered Architecture

Framework provides a developer with some tools and dictates how to use these tools properly.

Overview of Express

The code snippet below creates a simple Express server:

TEXT

```
const express = require('express');
const app = express();
app.listen(3000, () => {
  console.log('Server is started');
})
```

Expressjs provides us with three major features:

- routing
- middleware
- error handling

Routing

As you might have noticed from our previous example of using HTTP, it doesn't have any routing features. If you want to add a new endpoint you need manually check the HTTP method and URL of a request. Express provides quite a handy routing feature. For adding an endpoint you can just use `app(url, handler)`

method - is any HTTP request method, you can add an endpoint for. Or all it for all methods. url - is a string that represents URL of your endpoint handler - is a callback that will be called when a request with provided method and URL was made

TEXT

```
app.get('/users/:userId', () => {});
app.put('/users/:userId/groups', () => {});
app.post('/comments', () => {});
app.get('/comments', () => {});
app.delete('/comments/:commentId', () => {});
```

You might have noticed column: symbol in the path, that's route param. `/users/:userId` will match any route that starts with `/users/`.

Middlewares

Middleware - is a function that has 3 arguments: request, response, and next.

Middlewares can be used for some logic that should be executed for several or even all endpoints. For example, if you need to log information about every request, you can add a logger invocation in every handler, but it's much better to implement it only once.

Another use case is user authentication. You can create auth middleware that will reject all requests that don't have correct tokens in their headers. Example of logger middleware:

TEXT

```
const logger = (req: Request, res: Response, next: NextFunction) => {  
  console.log(`New request: ${req.method}, ${req.url}`);  
  next();  
}
```

To add middleware to all routes you can use use method of express instance:

TEXT

```
app.use(logger);
```

You can also specify paths when a middleware should be executed:

TEXT

```
app.use('/api', logger);
```

Express provides some built-in middlewares:

- `express.static`
- `express.json`
- `express.urlencoded`

Still there are plenty of open-source ones that you can use. Here is a small list of middlewares that Express team developed:

- `body-parser`
- `cookie-parser`
- `cors`
- `multer`
- `server-static`

Error handling

Express has its own error handling out of the box. If any of your middleware throws an exception, it will be caught by a built-in error handler and return an HTML string with a message and stack trace from an error object.

You can create error handler middleware that will be invoked only when there is an exception in middleware (or handlers) that were declared before the error handler. To convert a middleware into an error handler middleware, you need just to add one more argument at the beginning of the arguments list - `error`. Express distinguishes regular middleware from error handlers by the number of arguments that they have. For example:

TEXT

```
app.get('/api/users/:userId', () => {
  throw new Error('User was not found');
});

app.use((err, req, res, next) => {
  console.log(err);
  res.status(500);
  res.send({ message: err.message });
});
```

Layered architecture

Layered architecture in Express.js is a way of organizing your application code into distinct layers, where each layer has a specific responsibility. This design pattern helps with code readability, maintainability, scalability, and testability.

Routes Layer (Router)

- Responsibility: Maps HTTP requests (e.g. GET /users) to controller functions.
- Typical File(s): routes/userRoutes.js

Controller Layer

- Responsibility: Handles the request logic (e.g. parsing input, calling services).
- Typical File(s): controllers/userController.js

Model Layer (Optional if using ORM)

- Responsibility: Defines data structures or ORM schemas (e.g., using Mongoose, Sequelize).

Benefits of Layered Architecture

- Separation of concerns

- Reusability of logic (e.g., services can be reused by other interfaces like CLI or cron jobs)
- Easier testing (you can mock dependencies)
- Cleaner code organization

File System

A file is a container for data.

- It has a name and a location (path) in the file system
- It can contain any type of data - text, binary, images, etc.
- Files persist even after the program that created them has stopped running

TEXT

```
const fs = require('fs');
```

File System Module

This module allows you to:

- Read from files
- Write to files
- Update files
- Delete files
- Get file information

Synchronous and Asynchronous methods:

- Asynchronous (non-blocking) methods are preferred for performance
- Synchronous (blocking) methods end with "Sync" (e.g., readFileSync)

Read Files

The `fs.readFile()` method is used to read files on your computer.

TEXT

```
const fs = require('fs');
fs.readFile('file.txt', 'utf8', (err, data) => {
  if (err) {
    console.error("Error reading file:", err);
    return;
  }
  console.log("File content:", data);
});
```

Create Files

The File System module has methods for creating new files:

- `fs.appendFile()`
- `fs.writeFile()`
- `fs.open()`

`fs.appendFile()`

The `fs.appendFile()` method appends specified content to a file. If the file does not exist, the file will be created:

TEXT

```
fs.appendFile('mushroomList.txt', 'Champignon', (err) => {  
  if (err) throw err;  
  console.log('Saved!');  
});
```

Therefore, `mushroomList.txt` file will be created with the content `Champignon`.

`fs.writeFile()`

The `fs.writeFile()` method replaces the specified file and content if it exists. If the file does not exist, a new file, containing the specified content, will be created.

TEXT

```
const fs = require('fs');  
fs.writeFile('mushroomList.txt', 'Champignon', (err) => {  
  if (err) throw err;  
  console.log('Saved!');  
});
```

`fs.open()`

The `fs.open()` method in Node.js is used to open a file for reading, writing, or both.

TEXT

```
const fs = require('fs');  
fs.open('mushroomList.txt', 'r', (err, f) => {  
  console.log('Saved!');  
});
```

Update Files

The File System module has methods for updating files:

- `fs.appendFile()`
- `fs.writeFile()`

TEXT

```
const fs = require('fs');
fs.appendFile('mushroomList.txt', ', Shiitake', (err) => {
  if (err) throw err;
  console.log('Updated!');
});
```

The `fs.writeFile()` method replaces the specified file and content:

TEXT

```
const fs = require('fs');
fs.writeFile('mushroomList.txt', 'Enoki, Chanterelle', (err) => {
  if (err) throw err;
  console.log('Replaced!');
});
```

Delete Files

To delete a file with the File System module, use the `fs.unlink()` method.

TEXT

```
const fs = require('fs');
fs.unlink('mushroomList.txt', (err) => {
  if (err) throw err;
  console.log('File deleted!');
});
```

Rename Files

Use `fs.rename()` method for renaming a file.

TEXT

```
const fs = require('fs');
fs.rename('mushroomList.txt', 'fungusList.txt', (err) => {
  if (err) throw err;
  console.log('File Renamed!');
});
```

Three main ways to work with files

1. Promise-based (Async/Await)
2. Callback-based (Asynchronous)
3. Synchronous (Blocking)

1. Promise-based (Async/Await)

The File System Promises API is introduced to make file system operations easier to work with in modern JavaScript through the use of Promises and async/await syntax. Errors in promises can be caught using try...catch.

TEXT

```
const fs = require('fs').promises;

async function readFile() {
  try {
    const data = await fs.readFile('example.txt', 'utf-8');
    console.log(data);
  } catch (error) {
    console.error('Error reading file:', error);
  }
}

readFile();
```

2. Callback-based (Asynchronous)

The regular File System API uses callbacks to handle asynchronous file system operations.

Key Features:

- Callback-based: Requires you to pass a callback function to handle the result of asynchronous operations.
- Nested structure: Often leads to "callback hell" when handling multiple asynchronous operations.
- Error handling: Error-handling is done inside the callback, making it less intuitive compared to promises.

TEXT

```
const fs = require('fs');
fs.readFile('example.txt', 'utf-8', (error, data) => {
  if (error) {
    console.error('Error reading file:', error);
  } else {
    console.log(data);
  }
});
```

3. Synchronous (Blocking)

There's also the Synchronous File System API (fs with no callbacks or promises), which you can use for blocking operations. These methods block the event loop, so they're typically used only in scenarios where blocking behavior is acceptable.

TEXT

```
const fs = require('fs');
const data = fs.readFileSync('example.txt', 'utf-8');
console.log(data);
```

Error Handling in File Operations

File system operations can fail for various reasons, such as incorrect file paths, insufficient permissions, missing files, or system-related issues. Proper error handling ensures that your application reacts appropriately to these errors without crashing.

Always handle errors in async operations (callbacks, promises).

- Using Callbacks – Errors will be passed as the first parameter of the callback.
- Using Synchronous Methods – You can catch these errors using a try/catch block.
- Using Promises – try/catch with async/await

Types of Errors in File System Operations

- File Not Found: Attempting to read or delete a file that does not exist.
- Permission Denied: Lack of sufficient permissions to perform an operation (e.g., trying to write to a read-only file).
- Invalid Path: Providing an invalid or malformed file or directory path.
- File Lock: Trying to perform operations on a file that's in use by another process.
- Disk Space: Running out of disk space while writing to a file.

Error Codes in Node.js File System

- ENOENT: The file or directory does not exist.
- EACCES: Permission denied.
- EEXIST: File/directory already exists
- EISDIR: The operation was attempted on a directory instead of a file.
- EPERM: Operation not permitted.
- ENOTEMPTY: Directory not empty (when attempting to delete a non-empty directory).

File Permissions

Permission Types

- Read (r): Allows viewing/reading the file
- Write (w): Allows modifying the file
- Execute (x): Allows executing the file (for scripts/programs)

Permission Groups

- User (u): The owner of the file
- Group (g): Users who are members of the file's group
- Others (o): All other users

Permission Representation - Octal Notation

In Node.js, permissions are often represented as 3-digit octal numbers (base-8), where each digit represents permissions for user, group, and others respectively.

Each digit is a sum of:

- 4 = read
- 2 = write
- 1 = execute
- fs.chmod(): Used to change the permission of a file.
- fs.chown(): Used to change the owner and group of a file.

Examples:

- 0o755 (or 0755 in older Node versions): User: 7 (4+2+1 = read+write+execute)
Group: 5 (4+1 = read+execute) Others: 5 (4+1 = read+execute)
- 0o644: User: read+write Group: read Others: read

File descriptor

In Node.js, when you open a file (using fs.open), the function returns a file descriptor that can be used to perform further operations on the file.

A file descriptor serves as a unique identifier for an open file or I/O resource in the operating system, allowing programs to refer to and interact with that resource.

What are System Calls?

System calls are the fundamental interface between an application and the operating system kernel. They allow programs to request services from the OS like:

- File operations (open, read, write, close)
- Process control
- Device management
- Information maintenance
- Communication

Node.js Fundamentals

Node.js is an open-source, cross-platform JavaScript runtime environment that allows developers to execute JavaScript code outside of a web browser.

Since JavaScript is single-threaded, Node.js operates in a single process.

Non-blocking I/O model

Non-blocking I/O means that I/O operations (like reading files, querying a database, or making network requests) don't stop the execution of your program.

- When an I/O operation is initiated, Node.js doesn't wait for it to complete.
- Instead, it registers a callback (or uses Promises/async-await), and moves on to the next task.
- Once the I/O operation finishes, the callback is put in the event queue, and the event loop picks it up when it's time.

Node.js is single-threaded by default, but Worker Threads allow you to run CPU-intensive JavaScript operations in parallel without blocking the main event loop.

Why Use Worker Threads?

Node.js runs JavaScript in a single main thread (the Event Loop). Problem: Heavy computations (e.g., data processing, encryption, AI) block the main thread, making the app unresponsive. Solution: Worker Threads let you run CPU-heavy tasks in parallel without freezing the app.

The V8 JavaScript Engine

V8 is a high-performance JavaScript and WebAssembly engine developed by Google, written in C++. It's the core technology behind Node.js, Chrome, and Edge, primarily used to execute JavaScript code.

NVM (Node.js Version Manager)

Using nvm makes it easier to install and manage multiple versions of Node.js on a single local environment.

Project manifest (package.json)

A package.json file is a manifest of your project that contains metadata about the project, project's name, description, author, dependencies, scripts, and other settings

required to manage and run the application.

Create this file:

TEXT

```
npm init
```

Command above will ask you to answer some questions to specify base metadata of your project to generate package.json file.

TEXT

```
npm init -y
```

You can add -y flag in the end of the command to skip this questions and use default metadata for your project.

Scripts

The "scripts" section lets you define command-line shortcuts for tasks you frequently run — like starting your app, running tests, building assets, or restarting servers.

TEXT

```
npm start  
npm test
```

TEXT

```
"scripts": {  
  "start": "node index.js",  
  "dev": "nodemon index.js",  
  "test": "jest",  
  "build": "webpack --mode production",  
  "lint": "eslint ."  
}
```

Dependencies

Dependencies are the external packages your project needs to run properly in production.

To install all the dependencies from the package.json file you need to run the following command:

TEXT

```
npm install
```

Managing dependencies

- Regular dependency (runtime):

TEXT

```
npm install express
```

- Development-only dependency:

TEXT

```
npm install --save-dev nodemon
```

- Removing Dependencies

TEXT

```
npm uninstall nodemon
```

Dependencias útiles

Herramientas que facilitan el desarrollo en Node.js.

nodemon

Reinicia automáticamente la aplicación cuando detecta cambios en los archivos del proyecto.

TEXT

```
npm install -g nodemon
```

alternativa a nodemon (commando) sin instalar nodemon

Node.js tiene un flag nativo `--watch` que reinicia el proceso al detectar cambios.

TEXT

```
node --watch index.js
```

ESM (ECMAScript) modules

Si añades "type": "module" al archivo package.json, node.js cambia el comportamiento y comienza a tratar los archivos .js como módulos ECMAScript (ESM). Podrás usar la sintaxis moderna de import y export.

NPM COMMANDS

Comando	Descripción
<code>npm init</code>	inicializar proyecto con configuración
<code>npm init -y</code>	saltar configuración
<code>command + k</code>	limpiar la terminal
<code>npm list -g</code>	listar paquetes globales
<code>npm outdated</code>	listar paquetes desactualizados
<code>npm install express</code>	instalar dependencia
<code>npm install express@1.0.0</code>	instalar version especifica de dependencia
<code>npm install package1 package2</code>	instalar multiples paquetes de dependencias
<code>npm install express mongoose cors</code>	instalar multiples paquetes
<code>npm update slugify</code>	actualizar paquete
<code>npm uninstall express</code>	desinstalar dependencias
<code>npm install -g package1 package2</code>	instalar dependencia global
<code>npm install eslint prettier -D</code>	instalar dependencia de desarrollo
<code>npm install eslint --save-dev</code>	instalar dependencia de desarrollo
<code>npm install axios lodash -S eslint jest -D</code>	combinar dependencias
<code>-S</code> (or no flag) installs as dependencies.	
<code>-D</code> installs as devDependencies.	

NPX (Node Package Execute)

With npx you can execute your package without installation.

TEXT

```
npx <module>
```

Kill ports

TEXT

```
npx kill-port 4000
```

Variables de entorno

Las variables de entorno se acceden a través de `process.env` y se usan para almacenar configuración sensible o específica del entorno (como claves de API, credenciales de BD, etc.).

Imprimir variables de entorno

TEXT

```
console.log(process.env);
```

Graphql

GraphQL is a query language for APIs and a runtime for executing those queries. It was developed by Facebook and provides a flexible, efficient alternative to REST.

- Ask only for the data you need (no more, no less)
- Get everything in one request (even nested data)
- Strongly typed schema
- Enables real-time updates with subscriptions (optional)

Key Concepts in GraphQL

Schema

Defines types, queries, and mutations your API supports.

TEXT

```
type Book {  
  title: String  
  author: String  
}  
type Query {  
  books: [Book]  
}
```

Query

The client requests exactly the data it wants.

TEXT

```
{  
  books {  
    title  
  }  
}
```

Mutation

Used to modify data (like POST/PUT in REST).

TEXT

```
mutation {  
  addBook(title: "1984", author: "Orwell") {  
    title  
  }  
}
```

Resolvers

Functions that handle fetching the data for each field.

Types

Strongly-typed system: String, Int, Boolean, Float, ID, and custom types.

Installing GraphQL with Express.js

TEXT

```
npm install express graphql express-graphql
```

Create server.js

TEXT

```
const express = require('express');  
const { graphqlHTTP } = require('express-graphql');  
const { buildSchema } = require('graphql');  
  
const schema = buildSchema(`  
  type Query {  
    hello: String  
  }  
`);  
  
const root = {  
  hello: () => 'Hello world!',  
};  
  
const app = express();  
app.use('/graphql', graphqlHTTP({  
  schema,  
  rootValue: root,  
  graphiql: true,  
}));  
  
app.listen(4000, () => console.log('Server running on http://localhost:4000'));
```

Go to <http://localhost:4000/graphql> and try this query:

TEXT

```
{
  hello
}
```

GraphQL typically uses a single route

Why Only One Route? GraphQL is not REST. Instead of having multiple endpoints like:

- GET /users
- GET /users/:id
- POST /users

GraphQL uses one endpoint (e.g., /graphql) and the request body defines what to fetch or mutate.

typeDefs

typeDefs (short for type definitions) are used to define the schema of your GraphQL API — that is, the shape of the data and the operations (queries/mutations) that can be performed.

Jenkins

Jenkins is an open-source automation server used to build, test, and deploy software through CI/CD pipelines.

Key Concepts

- **Jobs** – A task or set of tasks Jenkins performs (build, test, deploy)
- **Pipelines** – Define the entire CI/CD workflow as code using a Jenkinsfile
- **Plugins** – Extend Jenkins functionality (Git, Docker, Slack, AWS, etc.)
- **Nodes/Agents** – Machines that execute jobs. The master distributes workload to agents
- **Jenkinsfile** – A text file that defines the pipeline as code, stored in the project repository

Basic Jenkinsfile Example

TEXT

```
pipeline {
  agent any
  stages {
    stage('Checkout') { steps { git 'https://github.com/user/repo.git' } }
    stage('Build')     { steps { sh 'npm install' } }
    stage('Test')      { steps { sh 'npm test' } }
    stage('Deploy')    { steps { sh 'npm run deploy' } }
  }
}
```

Installation

Download from jenkins.io and run:

TEXT

```
java -jar jenkins.war
```

Or install via Homebrew:

TEXT

```
brew install jenkins-lts  
brew services start jenkins-lts
```

Jenkins runs on `http://localhost:8080` by default.

Logging and debugging in Node.js

Debugging

Debugging is the process of identifying and fixing errors or issues in your code. It involves using various tools and techniques to examine the state of your program at runtime and track down the root cause of problems.

Terminology

Debugging has its own set of obscure jargon, such as the following:

- Breakpoint – When a debugger stops a program so that its state may be examined
- Debugger – A debugging tool that allows you to see internal variable states by running code line by line
- Feature – It's something that every developer says at some time in their career.
- Frequency – How frequently or under what circumstances a bug will appear
- It doesn't work – The most common but ineffective bug report
- Log point – A debugging instruction that displays the value of a variable at a certain point during execution
- Logging – Runtime data is written to the console or a file
- Logic error – The program works, but it doesn't do what it's supposed to do
- Priority – A bug is assigned to a list of upcoming updates
- Race condition – Bugs that are difficult to track are those that are reliant on the sequence or timing of uncontrollable events
- Refactoring – Rewriting code to make it easier to read and maintain
- Regression – A previously repaired bug reappears, possibly as a result of other updates
- Related – A bug that is linked to or comparable to another
- Reproduce – The steps that must be taken to cause the error
- RTFM error – A bug report disguised as user ignorance, usually followed by a response to "Read the Flipping Manual."
- Step into – Step inside the function being called when debugging code line by line
- Step out – Complete the current function's execution and return to the calling code when running line by line

- Step over – Complete the execution of a command without stepping into a function it calls when running line by line.
- Severity – The system's influence as a result of a bug
- Stack trace – The list of all methods that were called before the error
- Syntax error – Typographical errors, like `console.lug()`
- User error – An error created by a user rather than the application, although depending on that person's seniority, an update may still be required
- Watch – A variable to look at while the debugger is running
- Watchpoint – When a variable is set to a specified value, the program is stopped, similar to a breakpoint

Debugger statement

This statement allows you to pause the execution of your code at a particular point and examine the current state of your program.

TEXT

```
function addNumbers(a, b) {  
  debugger;  
  return a + b;  
}  
const result = addNumbers(2, 3);
```

Console methods

You can use the `console.log()` method to output information to the console at various points in your code to help you track down errors.

TEXT

```
function addNumbers(a, b) {  
  console.log(`Adding ${a} and ${b}...`);  
  const result = a + b;  
  console.log(`Result: ${result}`);  
  return result;  
}  
const result = addNumbers(2, 3);  
console.log(`Final result: ${result}`);
```

Node.js `util.debuglog`

The Node.js `util` module offers a built-in `debuglog` method which conditionally writes messages to `STDERR`:

TEXT

```
const util = require('util');  
const debuglog = util.debuglog('myapp');  
debuglog('myapp debug message [%d]', 123);
```

The debug module

The debug module in Node.js provides a simple and flexible logging utility that can be used to debug applications. This is a tiny, more advanced third-party alternative to `util.debuglog`.

Logging

Logging is the process of capturing events, messages, and data that occur during the operation of a software application or system. The purpose of logging is to provide a record of what happened in the system for diagnostic, monitoring, and analysis purposes.

- Debugging: logs can help you diagnose and troubleshoot issues that arise during development and production. With the right logging data, you can quickly identify the source of a problem and take steps to fix it.
- Monitoring: logs can be used to monitor the health and performance of your application in real-time. By analyzing your logs, you can detect and address potential issues before they become critical.
- Auditing: logs provide a record of the activity that occurs in your application, which can be useful for compliance and auditing purposes.

Libraries:

- Morgan
- Bunyan
- winston

Importaciones y exportaciones

ESM (ECMAScript) modules

TEXT

```
//package.json
{
  "name": "index",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "type": "module",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "express": "^4.17.1"
  }
}
```

Si añades "type": "module" al archivo package.json, node.js cambia el comportamiento y comienza a tratar los archivos .js como módulos ECMAScript (ESM). Podrás usar la sintaxis moderna de import y export.

TEXT

```
import fs from 'fs';
export function greet() {
  console.log('Hola Mundo');
}
```

CommonJS

Sin "type": "module": Los archivos .js se tratan como módulos CommonJS. Usas require para importar módulos y module.exports para exportar.

TEXT

```
const fs = require('fs');  
module.exports = function greet() {  
  console.log('Hola Mundo');  
};
```

Networking (pendiente)

HTTP(S)

Node.js has built-in support for HTTPS (HTTP Secure) through the https module. This allows you to create secure web servers and make secure requests to other servers.

TEXT

```
const http = require('http');
const https = require('https');
```

Running HTTP Web Server

The HTTP module can create an HTTP server that listens to server ports and gives a response back to the client.

TEXT

```
const http = require('http');
const port = 4000;
const server = http.createServer((req, res) => {
  res.end('Hello World');
});
server.listen(port, () => {
  console.log(`Server running at http://localhost:${port}/`);
});
// node app.js
// Server running at http://localhost:4000/
```

Network modules

http

The http module is a core module in Node.js that provides functionality for creating HTTP servers and making HTTP requests. It enables you to build networking applications, handle incoming requests, and communicate with other web servers.

Create HTTP server

TEXT

```
const http = require('http');
const server = http.createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');
  res.end('Hello, World!');
});
server.listen(3000, () => {
  console.log('Server is running on port 3000');
});
```

Make HTTP GET request

TEXT

```
const http = require('http');
const options = {
  hostname: 'www.google.com',
  port: 80,
  path: '/',
  method: 'GET',
};
const req = http.request(options, (res) => {
  console.log(`Status Code: ${res.statusCode}`);
  console.log('Response Headers:', res.headers);
  res.on('data', (chunk) => {
    console.log(`Received Data: ${chunk}`);
  });
  res.on('end', () => {
    console.log('Request Complete');
  });
});
req.on('error', (error) => {
  console.error(`Request Error: ${error.message}`);
});
req.end();
```

Process HTTP POST request

TEXT

```
const http = require('http');
const server = http.createServer((req, res) => {
  if (req.method === 'POST') {
    let body = '';
    req.on('data', (chunk) => {
      body += chunk;
    });
    req.on('end', () => {
      console.log(`Received Data: ${body}`);
      res.end('Data Received');
    });
  } else {
    res.statusCode = 404;
    res.end('Not Found');
  }
});
server.listen(3000, () => {
  console.log('Server is running on port 3000');
});
```

https

The https module extends the http module to provide support for HTTPS (HTTP over SSL/TLS) connections. It allows to create HTTPS server or make secure HTTPS requests.

Create HTTPS server

TEXT

```
const https = require('https');
const fs = require('fs');
const options = {
  key: fs.readFileSync('privatekey.pem'),
  cert: fs.readFileSync('certificate.pem'),
  ca: fs.readFileSync('ca.pem'),
  requestCert: true,
  rejectUnauthorized: true
};
const server = https.createServer(options, (req, res) => {
  const cert = req.socket.getPeerCertificate();
  console.log('Client Certificate:', cert);
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');
  res.end('Hello, World!');
});
server.listen(3000, () => {
  console.log('Server is running on port 3000');
});
```

Net

The net module provides API for creating TCP servers and establishing raw TCP connections. It is a lower-level module that can be used to implement custom protocols or handle specific network scenarios where higher-level modules like http or https are not suitable.

Create a TCP Server

TEXT

```
const net = require('net');
const server = net.createServer((socket) => {
  socket.on('data', (data) => {
    // Handle incoming data
  });
  socket.on('end', () => {
    // Handle connection termination
  });
});
server.listen(3000, () => {
  console.log('Server is running on port 3000');
});
```

Create a TCP Client

TEXT

```
const net = require('net');
const client = net.createConnection({ port: 3000 }, () => {
  console.log('Connected to server!');
  client.write('Hello, server!');
});
client.on('data', (data) => {
  console.log('Received data:', data.toString());
  client.end();
});
client.on('end', () => {
  console.log('Disconnected from server');
});
```

dns

The dns module provides API to perform DNS (Domain Name System) lookups and reverse lookups. It allows you to resolve domain names to IP addresses and vice versa.

DNS lookup

TEXT

```
const dns = require('dns');
dns.lookup('www.example.com', (err, address, family) => {
  if (err) {
    console.error(err);
    return;
  }
  console.log('IP Address:', address);
});
/*
IP Address: 93.184.216.34
*/
```

Resolve DNS

TEXT

```
const dns = require('dns');
dns.resolve('www.example.com', (err, addresses) => {
  if (err) {
    console.error(err);
    return;
  }
  console.log('IP Addresses:', addresses);
});
/*
IP Addresses: [ '93.184.216.34' ]
*/
```

url

The url module provides utilities for working with URLs. It allows you to parse, format, and manipulate URL strings.

URL Parsing

TEXT

```
const url = require('url');
const urlString = 'https://www.example.com:8080/path?param1=value1&param2=value2#fragment';
const parsedUrl = new URL(urlString);
console.log('Protocol:', parsedUrl.protocol);
console.log('Host:', parsedUrl.host);
console.log('Port:', parsedUrl.port);
console.log('Path:', parsedUrl.pathname);
console.log('Query:', parsedUrl.searchParams.toString());
console.log('Fragment:', parsedUrl.hash);
/*
Protocol: https:
Host: www.example.com:8080
Port: 8080
Path: /path
Query: param1=value1&param2=value2
Fragment: #fragment
*/
```

URL Formatting

TEXT

```
const url = require('url');
const urlObject = {
  protocol: 'https:',
  host: 'www.example.com',
  pathname: '/path',
  search: '?param1=value1&param2=value2',
  hash: '#fragment'
};
const formattedUrl = url.format(urlObject);
console.log('Formatted URL:', formattedUrl);
/*
Formatted URL: https://www.example.com/path?param1=value1&param2=value2#f
*/
```

Querystring

The querystring module provides utilities to parse and stringify query strings. It allows you to parse URL query parameters and generate query strings for HTTP requests.

TEXT

```
const querystring = require('querystring');
const queryString = 'param1=value1&param2=value2&param3=value3';
const parsedQuery = querystring.parse(queryString);
console.log('Parsed Query:', parsedQuery);
/*
Parsed Query: [Object: null prototype] {
  param1: 'value1',
  param2: 'value2',
  param3: 'value3'
}
*/
```

Stringify an Object into a Query String

TEXT

```
const querystring = require('querystring');
const obj = {
  param1: 'value1',
  param2: 'value2',
  param3: 'value3'
};
const queryString = querystring.stringify(obj);
console.log('Query String:', queryString);
/*
Query String: param1=value1&param2=value2&param3=value3
*/
```

Encode and decoding URL

TEXT

```
const querystring = require('querystring');
const encodedValue = querystring.escape('Special Characters: ?, =, &');
console.log('Encoded Value:', encodedValue);
const decodedValue = querystring.unescape(encodedValue);
console.log('Decoded Value:', decodedValue);
/*
Encoded Value: Special%20Characters%3A%20%3F%2C%20%3D%2C%20%26
Decoded Value: Special Characters: ?, =, &
*/
```

WebSockets

WebSockets - is a communication protocol that provides full-duplex, bidirectional communication between a client and a server over a single, long-lived connection. Unlike traditional HTTP, which follows a request-response model, WebSockets allow for real-time, interactive communication.

- **Persistent Connection:** WebSockets establish a persistent connection between the client and the server, allowing for continuous communication. Once the connection is established, both the client and server can send data to each other without the need for initiating new requests.
- **Full-Duplex Communication:** WebSockets support simultaneous bidirectional communication. This means that the client and server can send messages to each other independently and concurrently. It allows for real-time data streaming and instant updates.
- **Low Overhead:** The WebSocket protocol has a smaller overhead compared to HTTP. After the initial handshake, the subsequent messages have a minimal header size, reducing the amount of data transferred over the network.

- Event-Driven Model: WebSockets use an event-driven model where the client and server can trigger events and send messages to each other based on specific actions or conditions. This allows for efficient and reactive communication.

WebSockets are commonly used in various scenarios such as chat applications, real-time dashboards, and multiplayer games. Socket.io is a popular JavaScript library which simplifies the process of implementing real-time features in Node.js applications.

Operating System

The OS module in Node.js provides methods and properties for interacting with the operating system. It allows you to gather information about the system's hardware, operating system, current user environment, and more.

TEXT

```
const os = require('os');
```

It helps in:

- Getting system information (CPU, memory, OS type).
- Managing network interfaces.
- Handling user-related data.

Child Process

The `child_process` module in Node.js allows you to run shell commands, scripts, or other programs from your Node.js application.

- `stdout`: the output from the command
- `stderr`: errors from the command

exec() - Run a Shell Command

When you use `exec()`, Node collects all the output of a command into a buffer in memory, and only gives it to you when it's done. A buffer is a chunk of memory that temporarily holds data — in Node.js, a Buffer holds binary data. "Collect everything in a basket first... then hand it to me all at once."

TEXT

```
const childProcess = require('child_process');
const execProcess = (command) => {
  childProcess.exec(command, (error, stdout, stderr) => {
    console.log(`stdout: ${stdout}`);
    console.log(`stderr: ${stderr}`);
    if (error !== null) {
      console.log(`error: ${error}`);
    }
  });
}
execProcess('node -v');
// The output is the following:
// stdout: v16.14.0
// stderr:
```

execFile()

`execFile()` is a method from Node's `child_process` module that runs an executable file directly, without using a shell.

TEXT

```
// sayHello.js
console.log('Hello from another file!');

// main.js
const { execFile } = require('child_process');
execFile('node', ['sayHello.js'], (error, stdout, stderr) => {
  if (error) {
    console.error('Error:', error);
    return;
  }
  console.log('Output:', stdout); // Hello from another file!
});
```

spawn()

`spawn()` is a method in Node.js that lets you start a new process (run another command or program) and interact with it in real time. A stream lets you handle data piece by piece, as it comes in. When you use `spawn()`, it gives you a stream (`stdout`, `stderr`) — so you can listen for chunks of data and process them right away. "The water (data) flows in little by little — and I can start using it immediately."

TEXT

```
const { spawn } = require('child_process');
const python = spawn('python3', ['script.py', 'arg1', 'arg2']);

python.stdout.on('data', (data) => {
  console.log(`Python said: ${data}`);
});
python.stderr.on('data', (data) => {
  console.error(`Python error: ${data}`);
});
python.on('close', (code) => {
  console.log(`Python process exited with code ${code}`);
});
```

fork()

The `fork()` method is a specialized version of `spawn()` designed specifically for running Node.js scripts as child processes. where the parent and the child process can communicate with each other via `send()` (but child processes cannot communicate with each other).

TEXT

```
// index.js
const { fork } = require('child_process');
const child = fork('./child.js');

child.on('message', (message) => {
  console.log('Parent process received:', message);
});
child.send({ hello: 'from parent process' });

child.on('close', (code) => {
  console.log(`child process exited with code ${code}`);
});
```

TEXT

```
// child.js
process.on('message', (message) => {
  console.log('Child process received:', message);
});
process.send({ hello: 'from child process' });
```

The *Sync function

The functions `spawn`, `exec`, and `execFile` from the `child_process` module also have synchronous blocking versions that will wait until the child process exits.

TEXT

```
const {
  spawnSync,
  execSync,
  execFileSync,
} = require('child_process');
```

Cluster

Node.js runs on a single thread by default, which means it can only use one CPU core at a time.

The `Cluster` module allows you to:

- Take advantage of multiple CPU cores
- Run multiple instances of your app (called "workers")
- Share the same server port among them

TEXT

```
const cluster = require('cluster');
const http = require('http');
const numCPUs = require('os').cpus().length;

if (cluster.isMaster) {
  console.log(`Master ${process.pid} is running`);
  for (let i = 0; i < numCPUs; i++) {
    cluster.fork();
  }
  cluster.on('exit', (worker, code, signal) => {
    console.log(`worker ${worker.process.pid} died`);
    cluster.fork();
  });
} else {
  http.createServer((req, res) => {
    res.writeHead(200);
    res.end('Hello World\n');
  }).listen(8000);
  console.log(`Worker ${process.pid} started`);
}
```

How Clustering Works (Multiple Copies)

Your main application is duplicated one copy per CPU core by default.

Each copy:

- Runs in a separate process with its own memory
 - Has its own event loop and V8 instance
 - Shares the same port (e.g. all listen on port 3000)
 - Operates completely independently from others
-

Worker Threads

Worker Threads are a Node.js module (`worker_threads`) that enables the use of threads to execute JavaScript in parallel.

Why Use Worker Threads?

Node.js is single threaded by default which means CPU heavy tasks can block the event loop. Worker threads help by:

- Offloading CPU-intensive tasks from the main thread
- Improving performance for computational workloads
- Maintaining responsiveness of the main application
- Utilizing multiple CPU cores effectively

Key Concepts

- Isolation: Each worker has its own V8 instance, event loop, and memory.
- Communication: Workers communicate with the main thread via message passing.
- Shared Memory: You can share memory between threads using `SharedArrayBuffer`.
- No DOM: Unlike web workers, Node.js worker threads don't have access to browser APIs.

Process and a Thread

Process

An independent program execution unit with its own memory space. Created by the operating system when a program launches. Has its own: Memory (code, data, heap, stack), File descriptors, Security context (user permissions), Process ID (PID)

Thread

A thread is a lightweight unit of execution within a process. Threads share memory within the same process, allowing them to efficiently communicate and work concurrently.

Key Characteristics:

- Threads within a process use shared memory, making communication between threads faster.
- Node.js is inherently single threaded in its event loop, but there are worker threads available for multithreading.
- A thread is a smaller part inside a process. A process can have one or more threads, all working within the same memory space.

Think of it like this:

- A process is like an office with its own isolated space.
- Threads are employees inside that office who share the resources (like the office memory) and work together.

In Node.js: by default the event loop runs on a single thread (Node.js is single-threaded). However you can use worker threads for heavier tasks and parallel execution within the same process.

Cluster module = Multiple processes (good for web servers) Worker threads = Multiple threads in same process (good for CPU tasks)

Package.json

El archivo package.json es esencial en cualquier proyecto Node.js o que utilice el gestor de paquetes npm. Este archivo define la configuración del proyecto, sus dependencias y scripts, y permite que otras personas (o servidores) puedan trabajar con el proyecto de manera consistente.

Metadatos básicos del proyecto

name: Nombre del proyecto (único si se publica en npm). version: Versión del proyecto según el estándar SemVer. description: Breve descripción del proyecto. author: Información del autor o equipo (opcional). license: Licencia del proyecto (por ejemplo, MIT o ISC).

TEXT

```
{
  "name": "mi-proyecto",
  "version": "1.0.0",
  "description": "Un ejemplo básico de package.json",
  "author": "Alan",
  "license": "MIT"
}
```

Al crear un nuevo proyecto con npm init, se lanzará un asistente que tras algunas preguntas, crea un archivo llamado package.json en la carpeta raíz del proyecto, donde coloca toda la información que se conoce sobre el mismo.

TEXT

```
npm init -y
```

Este comando omitirá el asistente y creará el archivo package.json.

TEXT

```
{
  "name": "frontend-project",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}
```

Dependencias del proyecto

Dependencias normales (dependencies)

Estas son las librerías o módulos que tu aplicación necesita para ejecutarse en producción. Se especifican en el archivo package.json dentro de la sección "dependencies".

TEXT

```
npm install express
```

Resultado en package.json:

TEXT

```
"dependencies": {
  "express": "^4.18.2"
}
```

Dependencias de desarrollo (devDependencies)

Estas son librerías que solo se necesitan durante la etapa de desarrollo, no para que el programa funcione en producción. Se especifican en el archivo package.json dentro de "devDependencies". Ejemplo: Herramientas como eslint, jest o webpack son útiles para desarrollar, probar y compilar, pero no para ejecutar la aplicación final.

TEXT

```
npm install eslint --save-dev
```

Resultado en package.json:

TEXT

```
"devDependencies": {  
  "eslint": "^8.45.0"  
}
```

Dependencias globales

En Node.js, las dependencias globales son paquetes instalados con el comando `npm install -g`. Estas dependencias se instalan en el sistema de manera global, lo que significa que están disponibles desde cualquier lugar en tu máquina, sin necesidad de estar vinculadas a un proyecto específico.

Path module

The path module in Node.js is a built-in module that provides utilities to work with file and directory paths. It allows you to handle and manipulate file paths in a way that is platform-independent, meaning it works correctly on both Windows (which uses backslashes \) and Unix-like systems (which use forward slashes /). This ensures that your application can operate seamlessly across different platforms.

Common Scenarios Where the path Module is Useful:

- Joining or resolving path segments into a single path.
- Getting the extension of a file.
- Getting the directory name or file name from a path.
- Handling path formats across different operating systems (e.g., Windows vs. Linux).

Methods

path.join()

Joins multiple path segments into a single normalized path. It automatically handles platform-specific path separators.

TEXT

```
const filePath = path.join('folder', 'subfolder', 'file.txt');
console.log(filePath); // Outputs: "folder/subfolder/file.txt" (on Unix)
```

path.resolve()

Resolves a sequence of path segments into an absolute path. It starts from the current working directory unless you provide an absolute path.

TEXT

```
const absolutePath = path.resolve('folder', 'subfolder', 'file.txt');
console.log(absolutePath); // Outputs: "/absolute/path/to/folder/subfolder/file.txt"
```

path.basename()

Returns the last portion (the file name) of a path. It can also strip the file extension if specified.

TEXT

```
const filePath = '/folder/subfolder/file.txt';  
console.log(path.basename(filePath)); // Outputs: "file.txt"  
console.log(path.basename(filePath, '.txt')); // Outputs: "file"
```

path.extname()

Returns the extension (if any) of the file in a path.

TEXT

```
const filePath = '/folder/file.txt';  
console.log(path.extname(filePath)); // Outputs: ".txt"
```

Absolute Path VS Relative Path

When to Use Each?

Absolute Path

- When you need to reference a file or directory from anywhere in the file system.
- For scripts or files that rely on specific system locations.
- When unambiguity is required.

Relative Path

- When working on a project where file locations are relative to the project root folder or the current working directory.
- To improve portability and flexibility (e.g., your project can run in different environments without modification).
- To avoid hardcoding the absolute paths into your code.

Notes

Absolute Path An absolute path is the full path to a file or directory starting from the root of the file system.

mac: /usr/local/bin/node

linux: /home/user/documents/file.txt

windows: C:\Users\JohnDoe\Documents\file.txt

Relative Path A relative path is a way to specify the location of a file or directory in relation to your current working directory (or the directory in which a script is running).

- Depends on the current working directory.
- May use special notations like: . → Current directory. .. → Parent directory.

Relational Databases

A relational database (RDB) is type of database that stores a collection of data items with pre-defined relationships between them. These items are organized as a set of tables with columns and rows.

The rows in the table represent a record and contains all values from its columns. Each row in a table should be marked with some unique identifier called a primary key.

What is RDBMS

The software used to store, manage, query, and retrieve data stored in a relational database is called a relational database management system (RDBMS). The RDBMS provides an interface between users and applications and the database, as well as administrative functions for managing data storage, access, and performance.

- MySQL
- PostgreSQL
- Oracle
- SQL Server

Transactions and ACID

All database transactions in any RDBMS are ACID compliant or Atomic, Consistent, Isolated and Durable to ensure data integrity.

Atomicity defines that all operations in transaction treated as one "unit", which either succeeds completely or fails completely: if any of the statements constituting a transaction fails to complete, the entire transaction fails and the database is left unchanged. In previous example transaction consists of two operations with different bank accounts and its naturally that bank wants to roll back any changes if something goes wrong.

Consistency ensures that a transaction can only bring the database from one valid state to another. Database can have some constraints in tables, i.e. unique columns, cascades, triggers, foreign keys etc. and any data written to the database must be valid according to all defined rules. This prevents database corruption by an illegal transaction.

Isolation helps to isolate one transaction from another, because often operations are run simultaneously one transaction may affect results of another if changes are persisted immediately. In RDBMS changes usually can be seen only after transaction is finished,

so the effects of an incomplete transaction might not even be visible to other transactions.

Durability guarantees that once a transaction has been committed, it will remain committed even in the case of a system failure (e.g., power outage or crash). This usually means that completed transactions are recorded not in some cache but in non-volatile memory immediately, unlike from eventually consistent NoSql.

When do I need RDB for my project

SQL is a good choice when working with related, structured data. Relational databases are efficient, flexible and easily accessed by any application. A benefit of a relational database is that when one user updates a specific record, every instance of the database automatically refreshes, and that information is provided in real-time.

Data modelling

Normalisation

Normalisation is a database design technique that reduces data redundancy and eliminates undesirable characteristics like Insertion, Update and Deletion Anomalies. Normalisation rules divides larger tables into smaller tables and links them using relationships. The purpose of Normalisation in SQL is to eliminate redundant (repetitive) data and ensure data is stored logically. There are 6 normal forms, however, in most practical applications, normalisation achieves its best in 3rd Normal Form.

1 Normal form

To satisfy First normal form, each column of a table must have a single value. Columns which contain sets of values or nested records are not allowed, each record needs to be unique.

2 Normal form

If a table has a single column primary key, it automatically satisfies 2NF,

3 Normal form

The third normal form states that you should eliminate fields in a table that do not depend on the key.

- A Table is already in 2 NF
- Non-Primary key columns shouldn't depend on the other non-Primary key columns
- There is no transitive functional dependency.

Relationships

One-to-one A one-to-one (1:1) relationship means that each record in Table A relates to one, and only one, record in Table B, and each record in Table B relates to one, and only one, record in Table A.

One-to-many Relationship is also maintained by FK column in one of the tables.

Many-to-many A many-to-many relationship indicates that multiple records in a table are linked to multiple records in another table. Those records may only be associated with a single record (or none at all) but the key is that they can and often are linked to more than one. Many-to-many relation is implemented by creating a third table, known as a join table, and create one-to-many relationships between it and your two starting tables.

Indexes and keys

A KEY in SQL is a value used to identify records in a table uniquely. An SQL KEY is a single column or combination of multiple columns used to uniquely identify rows or tuples in the table. SQL Key is used to identify duplicate information, and it also helps establish a relationship between multiple tables in the database.

What is a Primary Key (or PK)?

A primary is a single column value used to identify a database record uniquely. It has following attributes:

- A primary key cannot be NULL
- A primary key value must be unique
- The primary key values should rarely be changed
- The primary key must be given a value when a new record is inserted.

What is Composite Key?

A composite key is a primary key composed of multiple columns used to identify a record uniquely.

What is Foreign Key?

We have already used Foreign Keys (FK) during normalization process to build relations. Foreign Key references the primary key of another Table! It helps connect your Tables.

- A foreign key can have a different name from its primary key
- It ensures rows in one table have corresponding rows in another
- Unlike the Primary key, they do not have to be unique. Most often they aren't
- Foreign keys can be null even though primary keys can not

Indexes

Indexing is the way to get an unordered table into an order that will maximize the query's efficiency while searching. When there is no index, and you try to search something in database its engine has to scan through every row to find all matching records. While nowadays hardware can be really performant still such operations are time-consuming, especially if you have millions and millions of rows. Looking through every single row is not very efficient.

SQL basics

SQL. Structured Query Language(SQL) is the database language by the use of which we can perform certain operations on the existing database, and also we can use this language to create a database. SQL uses certain commands like Create, Drop, Insert, etc. to carry out the required tasks.

These SQL commands are mainly categorized into four categories as:

- DDL – Data Definition Language
- DQL – Data Query Language
- DML – Data Manipulation Language
- DCL – Data Control Language

DDL (Data Definition Language)

DDL or Data Definition Language actually consists of the SQL commands that can be used to define the database schema. DDL is a set of SQL commands used to create, modify, and delete database structures but not data.

Create This command is used to create the database or its objects (like table, index, function, views, store procedure, and triggers).

TEXT

```
CREATE DATABASE test;
CREATE INDEX idx_name
ON Employee (Name);
CREATE TABLE [IF NOT EXISTS] table_name (
    column1 datatype(length) column_constraint,
    column2 datatype(length) column_constraint,
    column3 datatype(length) column_constraint,
    table_constraints
);
```

Drop DROP is a DDL command used to delete/remove the database objects from the SQL database. We can easily remove the entire table, view, or index from the database using this DDL command.

TEXT

```
DROP DATABASE database_name;  
DROP TABLE Employee;  
DROP INDEX index_name;
```

Alter ALTER is a DDL command which changes or modifies the existing structure of the database, and it also changes the schema of database objects. We can also add and drop constraints of the table using the ALTER command.

TEXT

```
ALTER TABLE Employee ADD Surname varchar;  
ALTER TABLE Employee DROP Surname;
```

DML (Data Manipulation Language)

DML is an abbreviation of Data Manipulation Language. We can easily access, store, modify, update and delete the existing records from the database using DML commands.

Following are the four main DML commands in SQL:

- SELECT Command
- INSERT Command
- UPDATE Command
- DELETE Command

Insert INSERT is a data manipulation command in SQL, which allows users to insert data in database tables and has next syntax.

TEXT

```
INSERT INTO TABLE_NAME ( column_Name1 , column_Name2 , column_Name3 , ...  
INSERT INTO Hardware ("Serial", os, ram) VALUES ('1dfg124sd2a', 'MAC', 32  
INSERT INTO Hardware ("Serial", os, ram) VALUES  
    ('1dfg124sd2a', 'MAC', 32),  
    ('aa45g5dd71a', 'Windows', 64),  
    ('gh26s2h61sd', 'Windows', 16);
```

Select SELECT is one of the most frequently used commands in SQL. The SELECT command shows the records of the specified table. It also shows the particular record of a particular column by using the WHERE clause.

TEXT

```
SELECT column_Name_1, column_Name_2, ..., column_Name_N FROM Name_of_table  
SELECT * from Table  
SELECT * from Hardware WHERE ram ≥ 16
```

Update UPDATE is a command in SQL, which allows users to update or modify the existing data in database tables and has next syntax:

TEXT

```
UPDATE Table_name SET [column_name1= value_1, ..., column_nameN = value_N  
UPDATE Employee SET office_id = 2 WHERE id = 2;
```

Delete DELETE is a DML command which allows SQL users to remove single or multiple existing records from the database tables. We use the WHERE clause with the DELETE command to select specific rows from the table. Syntax of DELETE Command:

TEXT

```
DELETE FROM Table_Name WHERE condition;
```

How to query relations

In lots of situations you will want to get information from several tables in one request. There is JOIN operator in SQL that is used in such situation. A JOIN clause is used to combine rows from two or more tables, based on a related column between them. There are 4 different types of the JOINS in SQL:

- (INNER) JOIN: Returns records that have matching values in both tables
- LEFT JOIN: Returns all records from the left table, and the matched records from the right table
- RIGHT JOIN: Returns all records from the right table, and the matched records from the left table
- FULL JOIN: Returns all records when there is a match in either left or right table

Node.js and database interactions

ORM

ORM solves most of these problems. Object-relational mapping lets people interact with databases using their programming languages rather than forcing them to use SQL. Developers can use tools called object-relational mappers. They show the data in a

structured way that helps users understand the content and layout of a database without using SQL. One of the major benefits is that it saves time compared to entering SQL queries.

MikroORM is the TypeScript ORM for Node.js based on Data Mapper, Unit of Work and Identity Map patterns.

Node.js REPL

The Node.js REPL is an interactive programming environment that allows you to execute JavaScript code without saving it to a file.

Read - Reads your input

Eval - Evaluates your JavaScript code

Print - Prints the result

Loop - Repeats the process until you exit

How to Start the Node.js REPL

Simply type node in your terminal/command prompt without any filename.

TEXT

```
$ node
Welcome to Node.js v18.12.1.
Type ".help" for more information.
>
```

The REPL is waiting for us to enter some JavaScript code, to be more precise.

Event Emitters

Event Emitters are a fundamental pattern in Node.js that enables objects to communicate asynchronously using events. They are at the core of Node's event-driven architecture. The EventEmitter is a core module that facilitates this communication between objects by allowing them to emit named events and listen (subscribe) to those events.

- Emit an event: Trigger an event with optional data.
- Listen for an event: Register a callback function that gets executed when the event is emitted.

TEXT

```
const EventEmitter = require('events');
const emitter = new EventEmitter();

emitter.on('greet', (name) => {
  console.log(`Hello, ${name}!`);
});

emitter.emit('greet', 'Alice'); // Output: Hello, Alice!
```

The EventEmitter in Node.js works with custom, user-defined events, not with operating system events or browser events.

What is an Event Emitter?

An Event Emitter is an object that can:

- Emit named events
- Register listener functions to be executed when specific events occur

This follows the publish-subscribe pattern where:

- Emitters "publish" (emit) events
- Listeners "subscribe" to events they're interested in

What kind of events?

EventEmitter handles asynchronous, custom events within a Node.js application. These events are simple string names that you define and control — you decide when to emit them (emit) and how to listen for them (on or once).

Common examples:

- Stream events like 'data', 'end', 'error'
- Server events like 'connection', 'disconnect'
- Custom application events like 'userRegistered', 'fileProcessed', 'taskCompleted'

What it's not:

- It's not for DOM events like 'click' or 'mouseover' (those are for browsers).
- It's not directly for OS signals (though Node's process can listen to things like SIGINT using EventEmitter).
- It's not related to Observables or Promises.

Real-world example

TEXT

```
// eventBus.js
const EventEmitter = require('events');
const eventBus = new EventEmitter();
module.exports = eventBus;

// listeners/email.js
const eventBus = require('../eventBus');
eventBus.on('userRegistered', (user) => {
  console.log(`📧 Sending welcome email to ${user.email}...`);
  setTimeout(() => {
    console.log(`Email sent to ${user.email}`);
  }, 1000);
});

// app.js
const express = require('express');
const eventBus = require('./eventBus');
require('./listeners/email');
const app = express();
app.use(express.json());
app.post('/register', (req, res) => {
  const user = req.body;
  console.log(`Registering user: ${user.name}`);
  eventBus.emit('userRegistered', user);
  res.status(201).send({ message: 'User registered successfully' });
});
app.listen(3000, () => console.log('Server running on http://localhost:3000'));
```

Creating an Event Emitter

TEXT

```
import { EventEmitter } from 'events';
const eventEmitter = new EventEmitter();
```

Below are the member functions in the EventEmitter class.

TEXT

```
class EventEmitter extends internal {
  static listenerCount(emitter: EventEmitter, event: string | symbol):
  static defaultMaxListeners: number;
  addListener(event: string | symbol, listener: (...arg: any[]) => void):
  on(event: string | symbol, listener: (...arg: any[]) => void): this;
  once(event: string | symbol, listener: (...arg: any[]) => void): this;
  prependListener(event: string | symbol, listener: (...arg: any[]) =>
  prependOnceListener(event: string | symbol, listener: (...arg: any[])
  off(event: string | symbol, listener: (...arg: any[]) => void): this;
  removeAllListeners(event?: string | symbol): this;
  setMaxListeners(n: number): this;
  getMaxListeners(): number;
  listeners(event: string | symbol): Function[];
  rawListeners(event: string | symbol): Function[];
  emit(event: string | symbol, ...arg: any[]): boolean;
  eventNames(): Array<string | symbol>;
  listenerCount(type: string | symbol): number;
}
```

.on() and .emit() of the EventEmitter

Emit events using .emit(eventName, [...args]) Listen to events using .on(eventName, callback)

TEXT

```
import { EventEmitter } from 'events';
const eventEmitter = new EventEmitter();

eventEmitter.on('myEvent', () => {
  console.log('Data Received');
});

eventEmitter.emit('myEvent');
// output: Data Received
```

EventEmitter Instance Should Be Singleton for a Single Event Name

In other words, the on() and the emit() functions must be called on the same EventEmitter instance. The listeners won't work if registered on a separate EventEmitter instance.

Event Emitter Execution Model: Synchronous by Default

Event emitters in Node.js are synchronous by default.

Order of Execution of the Listeners

TEXT

```
import { EventEmitter } from 'events';
const eventEmitter = new EventEmitter();

eventEmitter.on('myEvent', (data) => {
  console.log(data, '- FIRST');
});

console.log('Statement A');

eventEmitter.on("myEvent", data => {
  console.log(data, '- SECOND');
});

eventEmitter.emit('myEvent', 'Emitted Statement');
console.log("Statement B");

// output:
// Statement A
// Emitted Statement - FIRST
// Emitted Statement - SECOND
// Statement B
```

Note: once eventEmitters emits the event, listeners will be executed. That's why first prints Statement A, then the listeners and finally the Statement B.

Some Useful Member Functions of the EventEmitter Class

once()

With once(), the listener will be discarded after listening for an event. Events listened with once() will be triggered only once.

TEXT

```
import { EventEmitter } from "events";
const eventEmitter = new EventEmitter();

eventEmitter.on("myEvent", data => {
  console.log(data, "- ON");
});
eventEmitter.once("myEvent", data => {
  console.log(data, "- ONCE");
});

eventEmitter.emit("myEvent", "Emitted Statement");
eventEmitter.emit("myEvent", "Emitted Statement");
eventEmitter.emit("myEvent", "Emitted Statement");

// output:
// Emitted Statement - ON
// Emitted Statement - ONCE
// Emitted Statement - ON
// Emitted Statement - ON
```

Note: first, on and once are printed because the first emit triggers them. Then on, and finally on.

prependListener()

By default, event listeners are invoked in the order they are added. The `prependListener()` method can be used as an alternative to adding the event listener to the beginning of the listener array.

TEXT

```
import { EventEmitter } from "events";
const eventEmitter = new EventEmitter();

eventEmitter.on("myEvent", (arg) => console.log(arg, "- ON"));
eventEmitter.prependListener("myEvent", (arg) => console.log(arg, "- PREF"));

eventEmitter.emit("myEvent", "Emitted Statement");
console.log(eventEmitter.eventNames());
```

eventNames()

Get all the active event names.

removeListener()

This is used to remove a listener.

TEXT

```
import { EventEmitter } from "events";
const eventEmitter = new EventEmitter();

function func1(): void {
  console.log("EVENT TRIGGERED");
}

eventEmitter.on("myEvent", func1);
eventEmitter.on("myEvent2", func1);
console.log(eventEmitter.eventNames());

eventEmitter.removeListener("myEvent", func1);
console.log(eventEmitter.eventNames());

// Output
// [ 'myEvent', 'myEvent2' ]
// [ 'myEvent2' ]
```

To remove a listener, we need to pass the same function reference in the second parameter that was used to create a listener.

removeAllListeners()

This is used to remove all active event listeners from an EventEmitter instance.

TEXT

```
import { EventEmitter } from "events";
const eventEmitter = new EventEmitter();

function func1(): void {
  console.log("EVENT TRIGGERED");
}

eventEmitter.on("myEvent", func1);
eventEmitter.on("myEvent2", func1);
eventEmitter.removeAllListeners();
console.log(eventEmitter.eventNames());

// Output
// []
```

REST API

REST (REpresentational State Transfer) is an architectural style that defines a set of constraints to be used for creating web services.

Client-Server Architecture

- Separation of Concerns: The client and server have distinct responsibilities and concerns.
- Scalability: The client-server architecture allows independent scaling of client and server components. For example the server can be scaled up across multiple machines to handle increased load.
- Loose Coupling: The client and server are loosely coupled, meaning they can evolve and change independently without impacting each other. As long as the interface between the client and server remains consistent, updates or modifications to one component can be made without affecting the other.
- Interoperability: Clients and servers can be developed using different technologies or run on different platforms.

Stateless Interactions

Each request from a client to a server must contain all the necessary information for the server to understand and process the request. The server does not rely on any previous requests or stored client context to fulfill the current request.

Uniform interface

- Resource-Based: REST APIs expose resources, which are represented as nouns, such as "users", "products" or "orders" (pay attention that plural forms are used). Resources are identified by unique URLs. For example, /users might represent the collection of users, /users/123 - a specific user with the ID 123.
- CRUD Operations: The uniform interface provides standard operations (Create, Read, Update, Delete) that map to the HTTP methods: POST, GET, PUT/PATCH, and DELETE, respectively. These methods correspond to operations that can be performed on resources.
- Stateless Requests: Each request from the client to the server should contain all the necessary information for the server to understand and process the request. The server should not rely on any client-specific state or session.

- Hypermedia as the Engine of Application State (HATEOAS): REST APIs include hypermedia links in the response to provide navigation and discoverability. These links allow clients to discover related resources and available actions without prior knowledge. For example, a response for a user might include links to view the user's details, update the user, or retrieve related resources like the user's orders.

Cacheability

Responses from the server can be marked as cacheable or non-cacheable. If they are defined as cacheable ones, then the client can reuse the response data for equivalent requests in the future.

By default responses are not cached, meaning that the server needs to recompute it for each request. However, there are some cases when the response for a particular request may not change frequently or at all. In this case, responses can be stored in cache and served directly from it for subsequent identical requests. It allows to improve performance significantly by reducing the need for redundant processing and network round trips.

- Cache-Control: allows the server to specify caching directives to the client. Some common directives include:
 - max-age: Indicates the maximum time in seconds that the response can be cached before it is considered stale.
 - private: Specifies that the response is intended for a specific user and should not be cached by shared caches.
 - public: Indicates that the response can be cached by both private and shared caches.
 - no-cache: Indicates that the response should not be served from the cache.
- Expires: specifies date and time after which the response is considered stale and should not be used from the cache. It is often used together with the Cache-Control header to provide additional caching instructions.
- ETag: is used to uniquely identify a specific version of a resource representation.
- Last-Modified: indicates the date and time when the resource was last modified on the server.

Note: Cached responses should be invalidated or refreshed when the corresponding resources change to maintain data consistency

Layered system

REST allows the usage of intermediaries, such as proxies, gateways, or load balancers, between the client and the server.

These intermediaries can improve performance, security, and scalability. Each layer remains unaware of any internals of other layers, promoting loose coupling and system extensibility. For example, a client cannot tell whether it is connected directly to the end server or to an intermediary.

Code on demand

It allows the server to dynamically send executable code (such as JavaScript) to the client as part of the API response. The client can then execute this code to enhance its functionality or behavior.

This feature is typically used in scenarios where the server wants to provide additional functionality to the client without requiring prior knowledge or hardcoding everything into the client application.

For example, suppose a REST API provides a service that performs complex calculations or generates customized user interfaces. Instead of the client having to implement those functionalities themselves, the server can send executable code along with the response. The client then executes this code to perform the desired calculations or modify its user interface based on the provided logic.

Streams

Streams are collections of data. They are a way to work with data as it becomes available, piece by piece, instead of waiting for the entire dataset to load at once.

Streams are like pipes that let you read or write data piece by piece, instead of loading the whole thing into memory at once.

Streaming data is data that is continuously generated and sent in small pieces, often in real time.

Buffer

A buffer is a temporary storage area in memory used to hold chunks of data being passed back and forth in streams. Since streams process data asynchronously and incrementally, chunks of data need to be temporarily stored somewhere before being used. Buffers allow handling raw binary data (like text, images, or videos) directly without converting it.

Chunks are small manageable pieces of data that are read from a source or written to a destination in streams. Streams work on chunks instead of loading the entire data at once, which saves memory and boosts performance when working with large datasets or continuous data flows.

HighWaterMark

The HighWaterMark property in streams defines the maximum number of bytes (or chunks) that the buffer can hold before stopping temporary data flow. It acts as a threshold for the internal buffer.

1. In Readable Streams, it controls how much data can be read into memory (buffered) before emitting the data event and asking for more data.
2. In Writable Streams, it controls how much data can be stored in memory before telling the producer to stop and wait for space.

Backpressure

Backpressure occurs when writable streams receive more data than they are capable of processing. This mismatch can lead to resource overuse, memory issues, or even crashes if not properly managed.

Backpressure solution

To deal with backpressure in streams, the goal is to slow down the producer (readable stream) when the consumer (writable stream) is overwhelmed.

How to handle backpressure:

- Use the writable stream's `write()` method to check if more data can be written. It will return `false` when backpressure starts building up.
- Wait for the drain event to resume writing after backpressure is reduced.
- `pipe()` – This method handles backpressure automatically — if the writable stream can't handle the current pace of data, the readable stream pauses until the writable stream emits the drain event.
- Set `highWaterMark` to control buffer thresholds – Adjusting this value can help you manage backpressure effectively, especially for applications with specific needs (e.g., handling large chunks at once or limiting memory usage).
- Pipeline – This utility is a safer and more declarative way to manage streams and automatically handles backpressure. It also takes care of stream cleanup and error handling.

Types of Streams

Readable streams

These streams are used to read data from a source.

Other common uses of readable streams in Node.js applications are:

- Reading data from a file
- `process.stdin` - To read user input via `stdin` in a terminal application.
- `http.IncomingMessage` - To read an incoming request's content in an HTTP server or to read the server HTTP response in an HTTP client.

Writable streams

These streams are used to write data to a destination.

- Writing to a file, HTTP responses, or network sockets.
- `process.stdout` can be used to write data to standard output and is used internally by `console.log`.

Duplex streams

Duplex streams are both Readable and Writable, meaning they can read and write data simultaneously.

- The most common example of a duplex stream is `net.Socket`, used to read and write data to and from a socket.

Transform Streams

They are used to modify or transform data as it is read or written.

- zlib compression streams, crypto for encryption/decryption, or converting data formats.

How Transform Streams Work

- Receives data from a Readable Stream.
- Processes the data (e.g., converting, compressing, encrypting).
- Pushes the transformed data to the next Writable Stream.

Advantages working with streams

- Efficient memory usage - With streams, large amounts of data do not need to be loaded into memory, reducing the number of reads and write cycles required to perform operations.
- Better performance - With streams, there is higher data processing throughput since data is processed as soon as it becomes available rather than waiting for all the data to arrive and then process it.
- Increased composability - With streams, developers can compose complex applications that interconnect data between multiple pieces of code or even across applications. This benefit allows developers to build microservices with Node.js.
- Real-time applications - Streams are essential for creating real-time applications such as video streaming or chat applications

Example: Stream with backpressure handling

TEXT

```
const fs = require('fs');
const readableStream = fs.createReadStream('./archivo_grande.txt', {
  encoding: 'utf8',
  highWaterMark: 16 * 1024 // 16 KB (buffer size or chunk)
});
const writableStream = fs.createWriteStream('./archivo_copia.txt', {
  highWaterMark: 16 * 1024 // 16 KB
});

let bytesRead = 0;

readableStream.on('data', (chunk) => {
  console.log(`Read: ${chunk.length} bytes`);
  bytesRead += chunk.length;
  const canContinue = writableStream.write(chunk);
  if (!canContinue) {
    console.log('Buffer full, pausing reading...');
    readableStream.pause();
  }
});

writableStream.on('drain', () => {
  readableStream.resume();
});

readableStream.on('end', () => {
  console.log('reading complete. ');
  console.log(`Total of read bytes: ${bytesRead}`);
  writableStream.end();
});

readableStream.on('error', (err) => {
  console.error('Error en el stream de lectura:', err.message);
});
writableStream.on('error', (err) => {
  console.error('Error en el stream de escritura:', err.message);
});
writableStream.on('finish', () => {
  console.log('Escritura finalizada. ');
});
```

pipe()

The pipe method is a basic and widely used way to connect one stream to another stream. It allows you to transfer data from a Readable Stream to a Writable Stream, automatically managing the flow and backpressure.

TEXT

```
const fs = require('fs');
const readableStream = fs.createReadStream('./input.txt');
const writableStream = fs.createWriteStream('./output.txt');

readableStream.pipe(writableStream);

writableStream.on('finish', () => {
  console.log('Data successfully written to output.txt!');
});
```

The Stream module

Writable

You can use the Writable class to define custom behavior for handling written chunks of data.

TEXT

```
const { Writable } = require('stream');
const fs = require('fs');

class UpperCaseWritable extends Writable {
  constructor(outputFilePath, options) {
    super(options);
    this.fileStream = fs.createWriteStream(outputFilePath);
  }
  _write(chunk, encoding, callback) {
    const upperCaseData = chunk.toString().toUpperCase();
    this.fileStream.write(upperCaseData, callback);
  }
  _final(callback) {
    this.fileStream.end(() => {
      console.log('File write complete!');
      callback();
    });
  }
}

const outputFilePath = 'output.txt';
const upperCaseStream = new UpperCaseWritable(outputFilePath);

upperCaseStream.write('hello, ');
upperCaseStream.write('world!\n');
upperCaseStream.end(() => {
  console.log('Finished writing to the stream!');
});
```

Readable

The Readable class allows developers to define their own custom streams to handle specialized data sources in a memory-efficient, streaming manner.

TEXT

```
const { Readable } = require("node:stream");
const fs = require("node:fs");

class FileReadStream extends Readable {
  constructor({ highWaterMark, fileName }) {
    super({ highWaterMark });
    this.fileName = fileName;
    this.fd = null;
  }
  _construct(callback) {
    fs.open(this.fileName, "r", (err, fd) => {
      if (err) return callback(err);
      this.fd = fd;
      callback();
    });
  }
  _read(size) {
    const buff = Buffer.alloc(size);
    fs.read(this.fd, buff, 0, size, null, (err, bytesRead) => {
      if (err) return this.destroy(err);
      this.push(bytesRead > 0 ? buff.subarray(0, 12) : null);
    });
  }
  _destroy(error, callback) {
    if (this.fd) {
      fs.close(this.fd, (err) => callback(err || error));
    } else {
      callback(error);
    }
  }
}

const stream = new FileReadStream({ fileName: "text.txt" });
stream.on("data", (chunk) => {
  console.log(chunk.length);
  console.log(chunk.toString("utf-8"));
});
stream.on("end", () => {
  console.log("Stream is done reading.");
});
```

pipeline()

A pipeline refers to chaining multiple streams together in a sequential manner such that the output of one stream serves as the input to the next.

Key Benefits of pipeline:

- Automatic Error Handling: It listens for errors on all streams and destroys them if any stream fails.
- Cleaner Syntax: It removes the boilerplate code required for chaining streams manually.
- Resource Management: It ensures that streams are closed/cleaned up properly.

File Compression Pipeline (Working Example)

TEXT

```
const fs = require('fs');
const zlib = require('zlib');
const { pipeline } = require('stream/promises');

async function compressFile() {
  try {
    await pipeline(
      fs.createReadStream('input.txt'),
      zlib.createGzip(),
      fs.createWriteStream('output.txt.gz')
    );
    console.log('Pipeline succeeded - file compressed!');
  } catch (err) {
    console.error('Pipeline failed:', err);
  }
}

compressFile();
```

pipe vs pipeline

Pipe (disadvantages): If something goes wrong (e.g., reading fails, compression throws an error), you need to manually handle errors for each stream. You must also manually handle cleanup (e.g., closing streams).

Pipeline (advantages): Errors in any stream in the chain are automatically handled. All streams are cleaned up (destroyed) to ensure there are no resource leaks.

Pipe example – handling errors

TEXT

```
const fs = require('fs');
const zlib = require('zlib');

const readableStream = fs.createReadStream('input.txt');
const gzipStream = zlib.createGzip();
const writableStream = fs.createWriteStream('output.txt.gz');

readableStream
  .pipe(gzipStream)
  .pipe(writableStream)
  .on('finish', () => {
    console.log('Compression succeeded!');
  })
  .on('error', (err) => {
    console.error('Pipeline failed:', err);
  });
```

Pipeline example – handling errors

TEXT

```
const fs = require('fs');
const zlib = require('zlib');
const { pipeline } = require('stream');

const readableStream = fs.createReadStream('input.txt');
const gzipStream = zlib.createGzip();
const writableStream = fs.createWriteStream('output.txt.gz');

pipeline(
  readableStream,
  gzipStream,
  writableStream,
  (err) => {
    if (err) {
      console.error('Pipeline failed:', err);
    } else {
      console.log('Pipeline succeeded!');
    }
  }
);
```

Duplex Streams

A Duplex stream is a combination of both a readable and a writable stream. It allows two-way communication, meaning you can both read from and write to the same

stream.

TEXT

```
const { Duplex } = require('stream');

class MyDuplexStream extends Duplex {
  constructor() {
    super();
    this.data = [];
  }
  _write(chunk, encoding, callback) {
    console.log(`Writing: ${chunk.toString()}`);
    this.data.push(chunk);
    callback();
  }
  _read(size) {
    if (this.data.length > 0) {
      this.push(this.data.shift());
    } else {
      this.push(null);
    }
  }
}

const myDuplex = new MyDuplexStream();
myDuplex.write('Hello World');
myDuplex.write('Node.js');
myDuplex.end();
myDuplex.on('data', (chunk) => {
  console.log(`Reading: ${chunk.toString()}`);
});
```

Transform Streams

A Transform stream is a special type of Duplex stream specifically designed for modifying ("transforming") the data as it passes through.

TEXT

```
const { Transform } = require('stream');

class UpperCaseTransform extends Transform {
  _transform(chunk, encoding, callback) {
    this.push(chunk.toString().toUpperCase());
    callback();
  }
}

const upperCaseStream = new UpperCaseTransform();
process.stdin.pipe(upperCaseStream).pipe(process.stdout);
```

Testing

Writing tests helps you to:

1. build confidence in how your code works
2. reduce potential bugs & deliver new changes quickly
3. receive instant feedback & catch regression early
4. refactor, deploy and release with higher confidence

Unit tests

Unit tests check the logic of individual functions or units in isolation. They are at the base of the Testing Pyramid because they take the least time and cost.

Unit tests should be:

- Fast: quick to be run and see the results
- Isolated: should not be influenced by any external factors
- Repeatable: output shouldn't change based on being run on different environments
- Self-validating: no manual work needed to check if test passed or not
- Thorough: should cover happy and error paths, edge cases, large input values, etc

Example: you have a function `sum()` that calculates the sum of two numbers. You want to test if `2+2` equals `4`, `2+3` equals `5`, etc.

Integration tests

Integration tests validate the interaction of a piece of code with external components. For this reason, they run slower than unit tests.

Example: you have two modules `Users` and `Groups`. `Users` calls `Groups` to retrieve user groups information. You want to test the integration between these two modules.

E2E tests

E2E tests are at the top of the Testing Pyramid because they involve the largest scope and are the most complex.

- for frontend applications, E2E tests are UI tests where the actual user interaction is checked in combination with API.
- for backend applications, E2E tests are more like API tests, where we test how our API is responding to requests.

They take the longest to run as they replicate the way an actual user or service interaction and check all the system layers.

Example: you have an endpoint POST /users to create users. You want to test API status codes, response structure, etc.

Testing libraries in Node.js

- Jest
- Mocha Mocha is a little more complicated than Jest when it comes to ease of use. Jest is designed to be simple and straightforward, while Mocha has more options and can be more difficult to learn.

Getting started with Jest

Installation

TEXT

```
npm install --save-dev jest
```

package.json

TEXT

```
{
  "scripts": {
    "test": "jest"
  }
}
```

jest.config.js

TEXT

```
module.exports = {
  testEnvironment: 'node',
  roots: ['./src'],
};
```

Typescript support

TEXT

```
npm install --save-dev ts-jest @types/jest
```

jest.config.js

TEXT

```
module.exports = {  
  preset: 'ts-jest',  
};
```

Jest core concepts

Grouping tests

Both functions do exactly the same thing

TEXT

```
// example with test()  
test('adds 5 + 2 to equal 7', () => {  
  expect(sum(5, 2)).toBe(7);  
});  
// example with it()  
it('adds 5 + 2 to equal 7', () => {  
  expect(sum(5, 2)).toBe(7);  
});
```

Several tests can be grouped using describe().

TEXT

```
describe('Create user', () => {  
  it('should create valid user object', () => { ... })  
  it('should throw error if username & password are not provided', () => { ... })  
})
```

Jest Test Lifecycle

The Four Main Lifecycle Methods

- beforeAll() Runs once before all tests in a test suite (describe block) Used for one-time setup (e.g., database connection)
- afterAll() Runs once after all tests complete Used for cleanup (e.g., closing database connection)
- beforeEach() Runs before each individual test Used to reset state between tests
- afterEach() Runs after each individual test Used for cleanup after each test

Mocks

Mocks are a way to isolate the code you're testing by replacing dependencies (like functions, modules, or entire libraries) with fake versions that you control.

Real API

TEXT

```
import axios from 'axios';
const API_URL = 'https://api.genderize.io';

type GenderResponse = {
  name: string;
  gender: string;
  probability: number;
  count: number;
};

const getGenderByName = async (name: string): Promise<GenderResponse | Error> => {
  const { data: gender } = await axios.get < GenderResponse > (`${API_URL}/name/${name}`);
  return gender;
};

export default getGenderByName;
```

Mock

TEXT

```
import axios from 'axios';
import getGenderByName from './get-gender-by-name';

const GENDER_JOHN = {
  name: 'john',
  gender: 'male',
  probability: 0.999,
  count: 1,
};

jest.mock('axios');

const mockedAxios = axios as jest.Mocked<typeof axios>;

describe('[Mocks] Get gender by name', () => {
  test('should return gender', async () => {
    mockedAxios.get.mockResolvedValue({ data: GENDER_JOHN });
    const genderResponse = await getGenderByName(GENDER_JOHN.name);
    expect(genderResponse).toEqual(GENDER_JOHN);
  });
  afterEach(() => {
    jest.clearAllMocks();
  });
});
```

jest.spyOn()

jest.spyOn() is like hiring a detective to watch a specific function and report back to you about:

- Was the function called?
- How many times?
- With what arguments?
- What did it return?

But here's the key part: the original function keeps working normally unless you tell the detective to interfere.

TEXT

```
import axios from 'axios';
import getGenderByName from './get-gender-by-name';

const GENDER_JOHN = {
  name: 'john',
  gender: 'male',
  probability: 0.999,
  count: 1,
};

describe('[Spies] Get gender by name', () => {
  test('should return gender', async () => {
    jest.spyOn(axios, 'get').mockImplementation(() => Promise.resolve({}));
    const genderResponse = await getGenderByName(GENDER_JOHN.name);
    expect(genderResponse).toEqual(GENDER_JOHN);
  });
  afterEach(() => {
    jest.clearAllMocks();
  });
});
```

Code Coverage

Code coverage is a metric that can help you understand how much of your source is tested.

- Shows untested parts of your code
- Helps catch hidden bugs
- Improves code quality
- Encourages writing more complete tests

Types of Code Coverage

- Line Coverage – Did each line of code run?
- Branch Coverage – Did every if/else or switch path get tested?
- Function Coverage – Was each function or method called?
- Statement Coverage – Were all statements executed?

How It Works

1. A coverage tool adds trackers into your code (this is called instrumentation).
2. You run your tests.
3. The tool watches which parts of the code were executed.

4. It creates a report showing what was tested (and what wasn't).